
Aplicacions Distribuïdes

Silvia Llorente
silvia.llorente@upc.edu

Distributed Multimedia Applications Group (DMAG)
Departament d'Arquitectura de Computadors (DAC)

Introduction

- Application Level
- Client / Server Model (C/S)
- Historical operations and methods for remote invocation
 - RPC, RMI, CORBA

Introduction

- Communication libraries
 - Sockets: Transport level TCP, UDP (XC)
 - HTTP Protocol
- Information formats
 - MIME
 - HTML y HTML5
 - XML

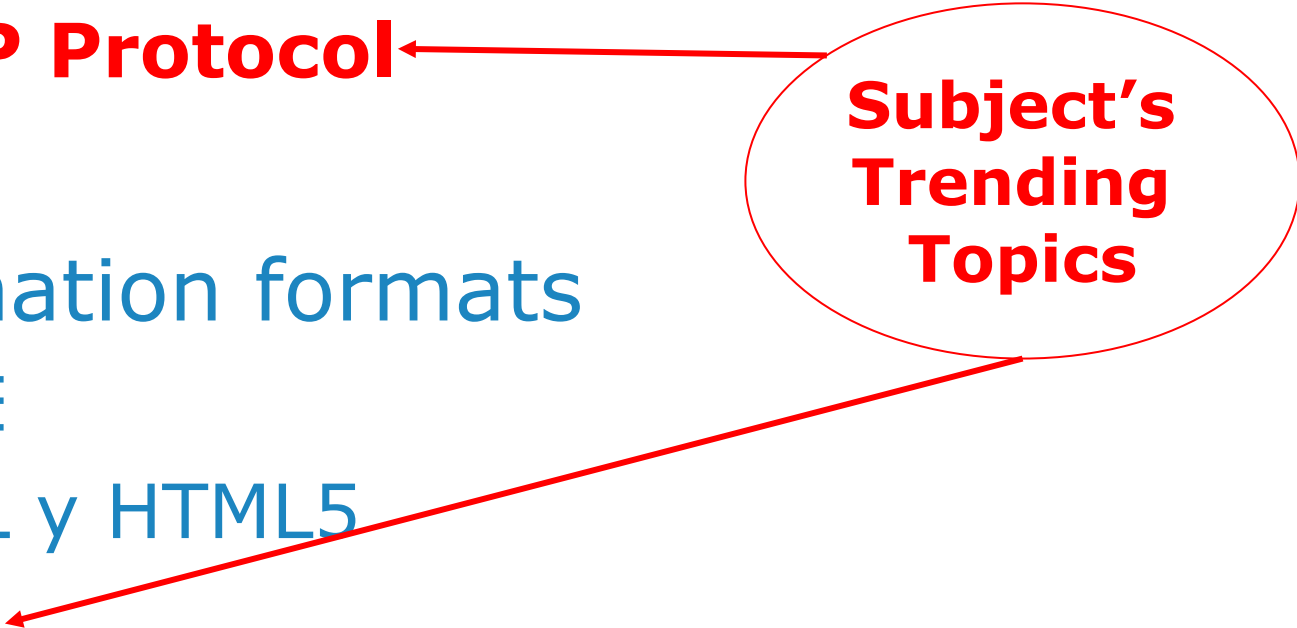
Introduction

- Communication libraries
 - Sockets: Transport level TCP, UDP (XC)
 - **HTTP Protocol**
- Information formats
 - MIME
 - HTML y HTML5
 - **XML**

Introduction

- Communication libraries
 - Sockets: Transport level TCP, UDP (XC)
 - **HTTP Protocol**
- Information formats
 - MIME
 - HTML y HTML5
 - **XML**

**Subject's
Trending
Topics**



Networks: Historical perspective



Phone networks for voice transmission

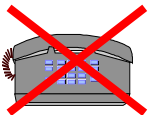
60's



First commercial computers



Modems for data transmission in phone networks



Phone networks too expensive for sending data, data networks appear

Networks: Historical perspective



Computer manufacturing companies like IBM or DEC provide everything required by their clients, from wires to OS

Closed Systems

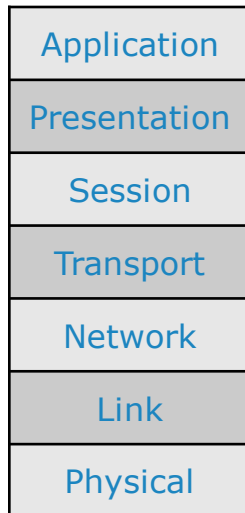
70's

Ethernet Protocol for LAN's
UNIX appears
TCP/IP Protocols

Open Systems

Networks: Historical perspective

ISO (*International Organization for Standardization*) and CCITT (Consultative Committee for International Telegraphy and Telephony) propose OSI (*Open Systems Interconnection*) protocol stack



Harmonize different alternatives (proprietary, open, ...) with a model organized in levels

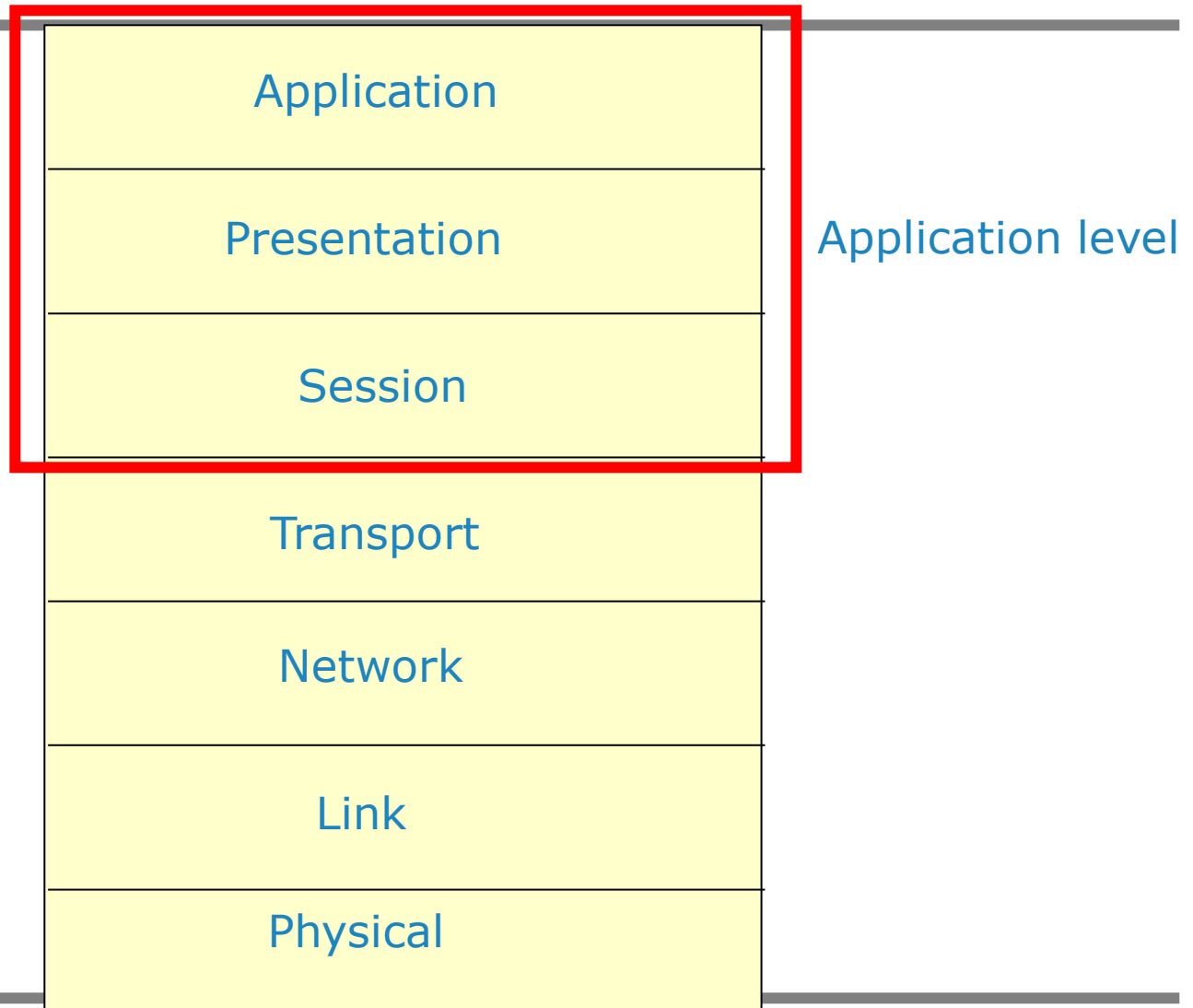
Networks: Historical perspective

Nowadays

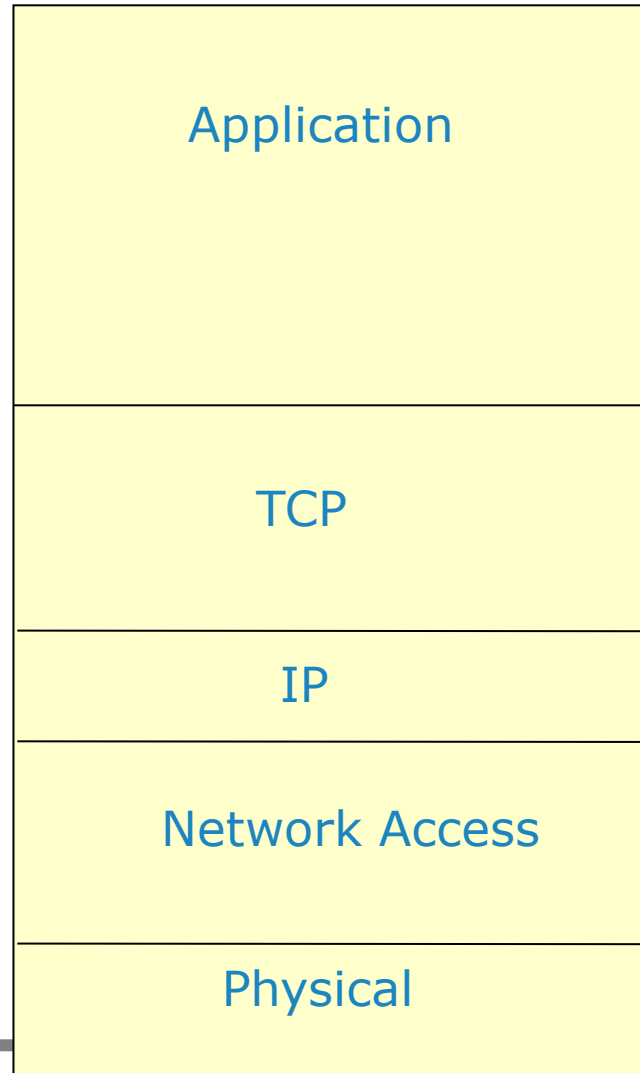
Internet model based on TCP/IP has succeed and Client/Server applications like WWW (*World Wide Web*) are the most used, although there is an evolution to mobile applications (also based on TCP/IP)

OSI model is not really used as it is too complex, but it is useful to describe protocol architectures

OSI model



Internet model



Internet Model vs. OSI Model

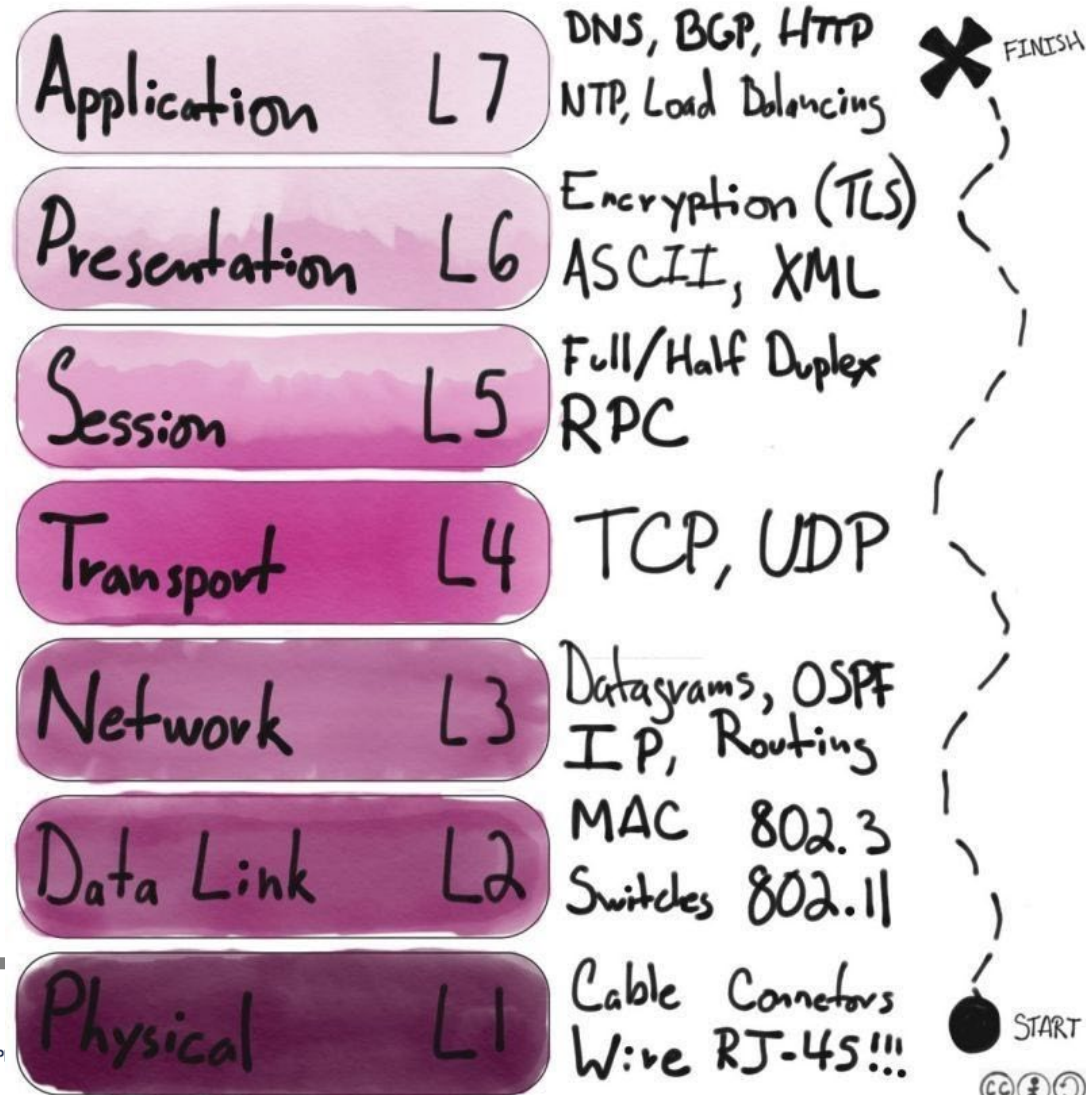
Application	Application
	Presentation
	Session
TCP	Transport
IP	Network
Network Access	Link
Physical	Physical

Source: Communications and computer networks, 6^a ed., William Stallings

OSI Model

@Chris Short
chrisshort.net

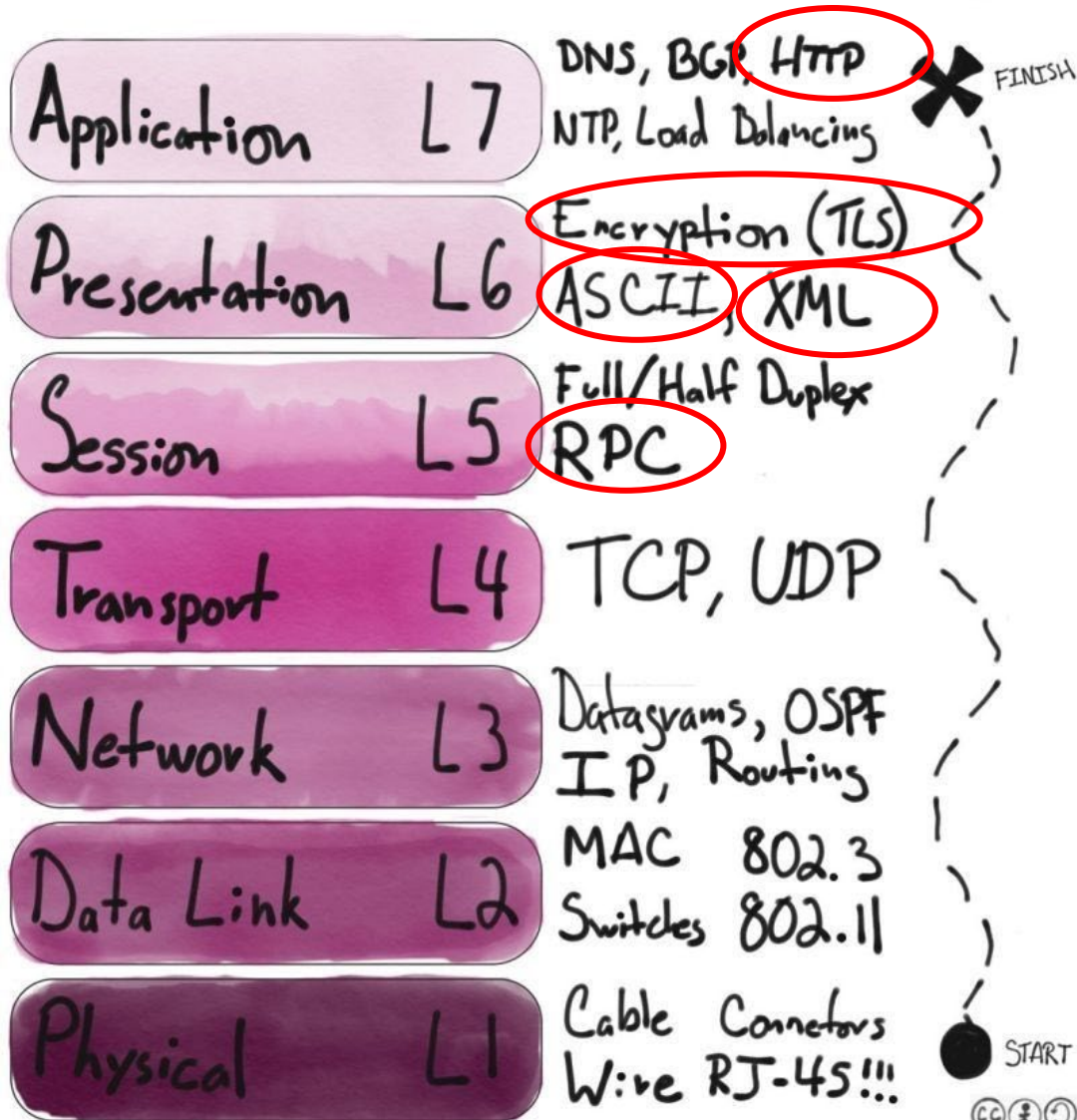
★ How Engineers Troubleshoot ***★***



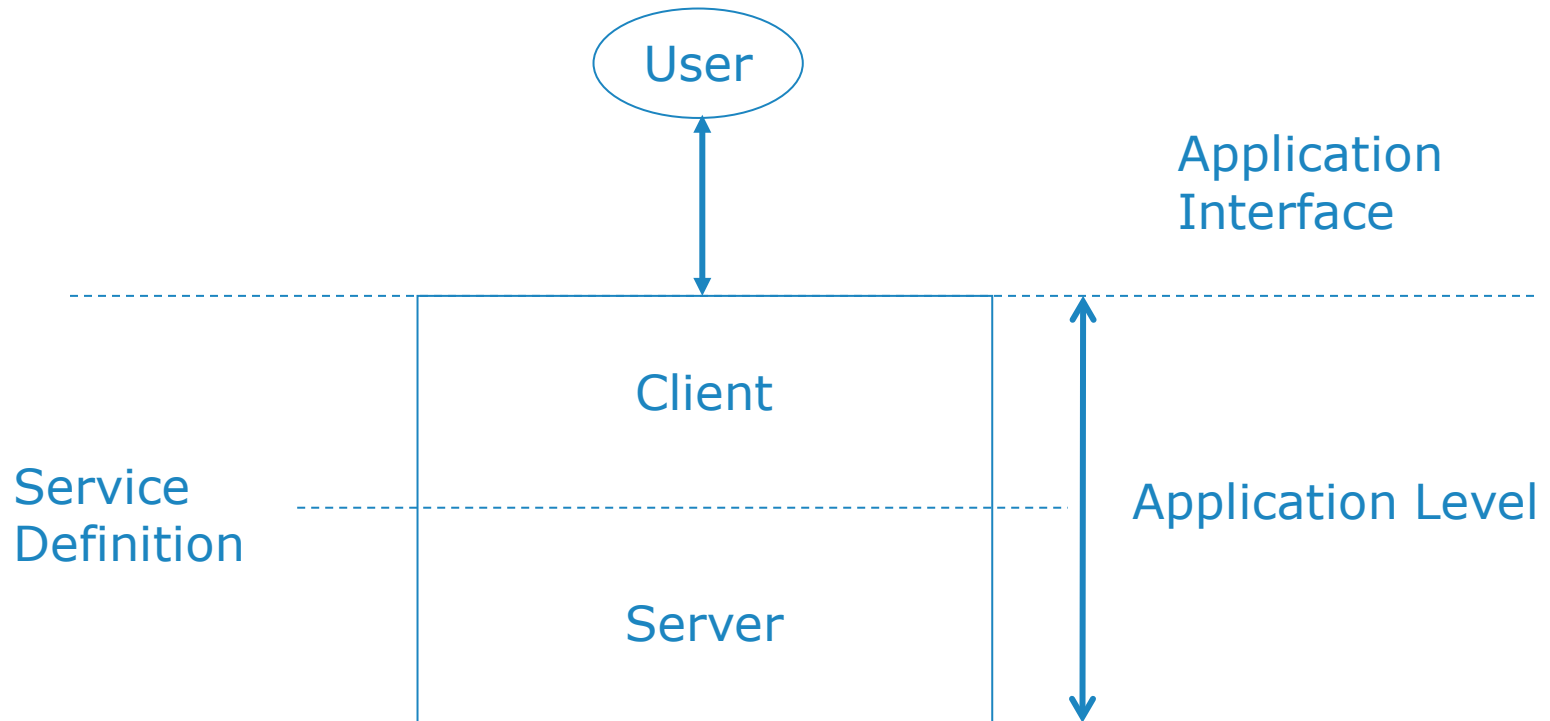
OSI Model

@Chris Short
chrisshort.net

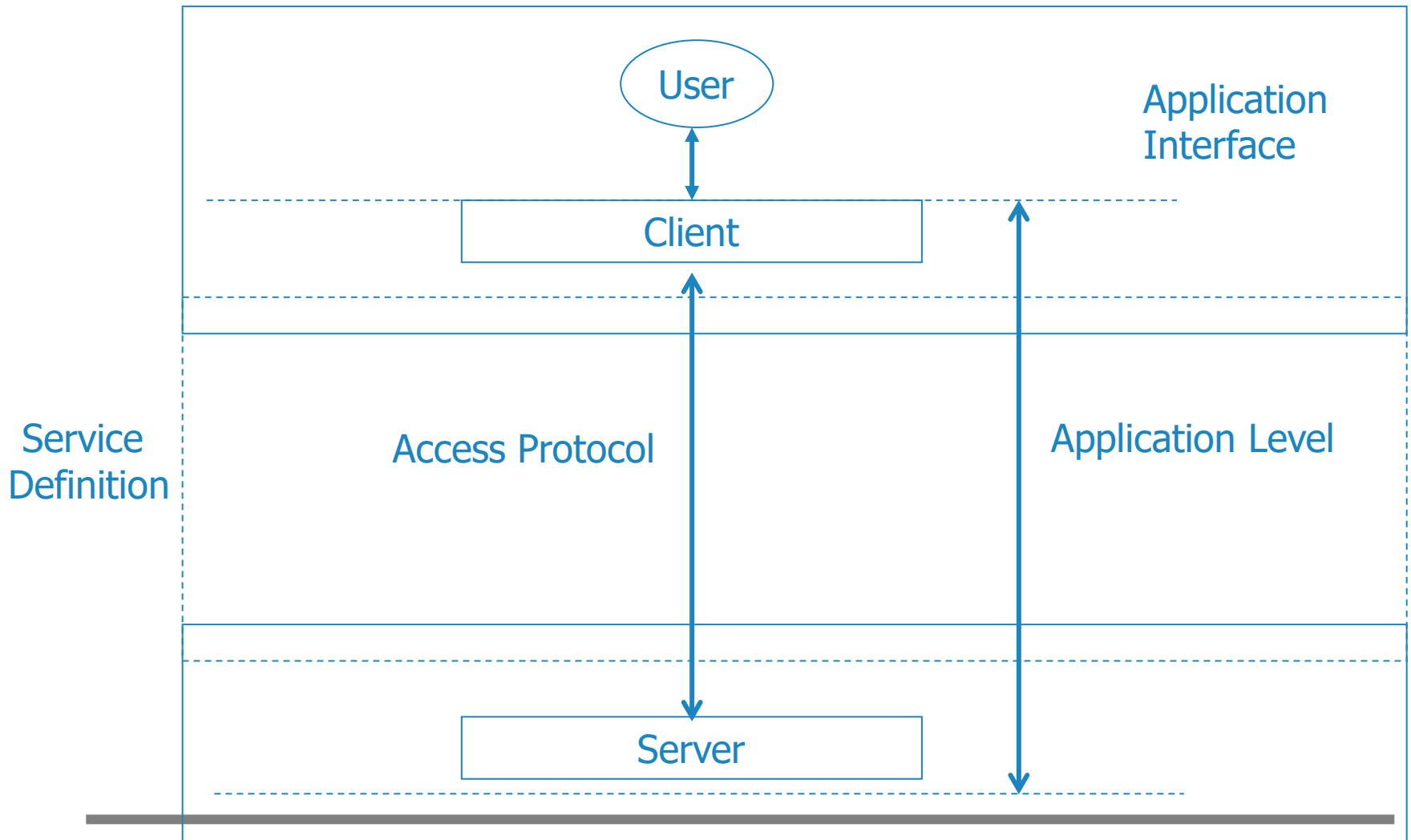
★ How Engineers Troubleshoot ***★***



Client / Server model (C/S)



Client / Server model (C/S)



C/S model - Client

- Usually in the user machine
- Main functionality:
 - Convert local calls to remote ones
 - Receive results and give them to the user

C/S model - Server

- Usually in a machine different from client's one, but it could be the same (we will see in the lab)
- Main functionality:
 - Solve operations invoked by user
 - Returns operation results

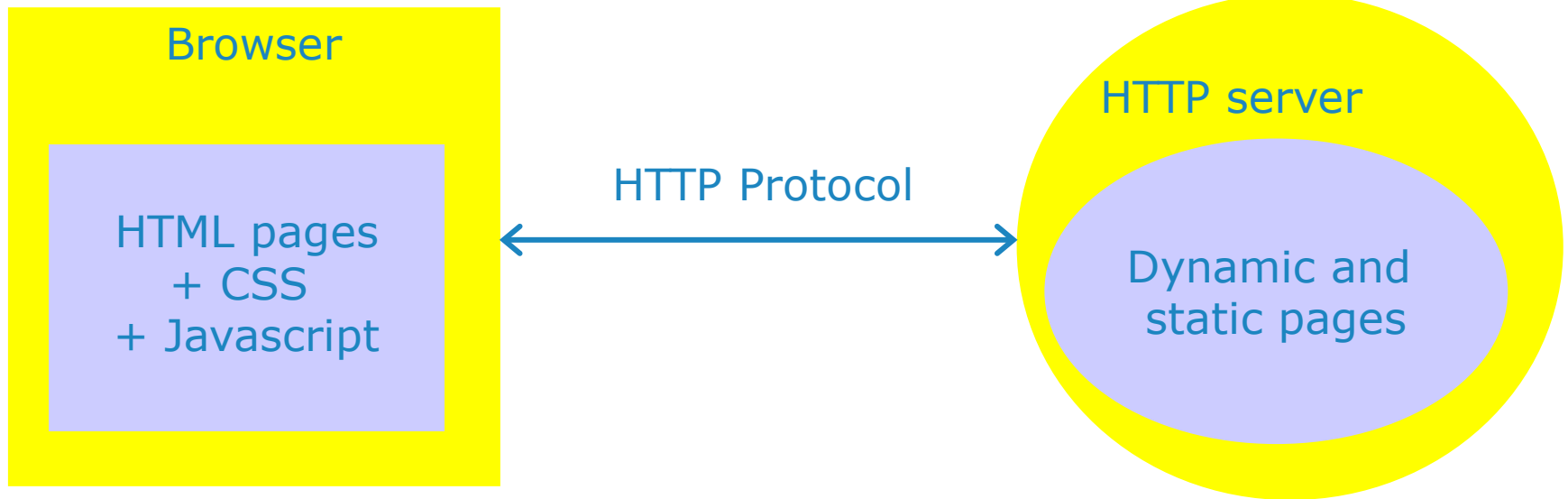
Distributed Application Example

- Racó de la FIB
- Client / Server Application
- Makes use of HTTP protocol to communicate

Architecture

Client

Server



Client Application

- Browser
- Application level protocol: HTTP
- Interprets and shows files in HTML format (also other file formats, like images)
- HTML files may contain forms
 - Can be combined with other file types

Client Application

- Communication protocol: HTTP
- Command based
- Allows access to server hosted resources
 - Not only HTML static files, also other types of files or HTML files created dynamically
- Commands example: GET and POST
 - Resource access
 - Also allow sending information to the server

Server application

- Process that receives HTTP commands and decides what to do
- If the requested resource is a servlet → The Servlet container comes into play → *Topic 2*

Distributed Application Definition

- An application made up of distinct components running in separate runtime environments, usually on different platforms connected via a network
- Typical distributed applications are two-tier (client-server), three-tier (client-middleware-server) and multitier (client-multiple middleware-multiple servers)

Operation and method remote invocation

- A little bit of history...

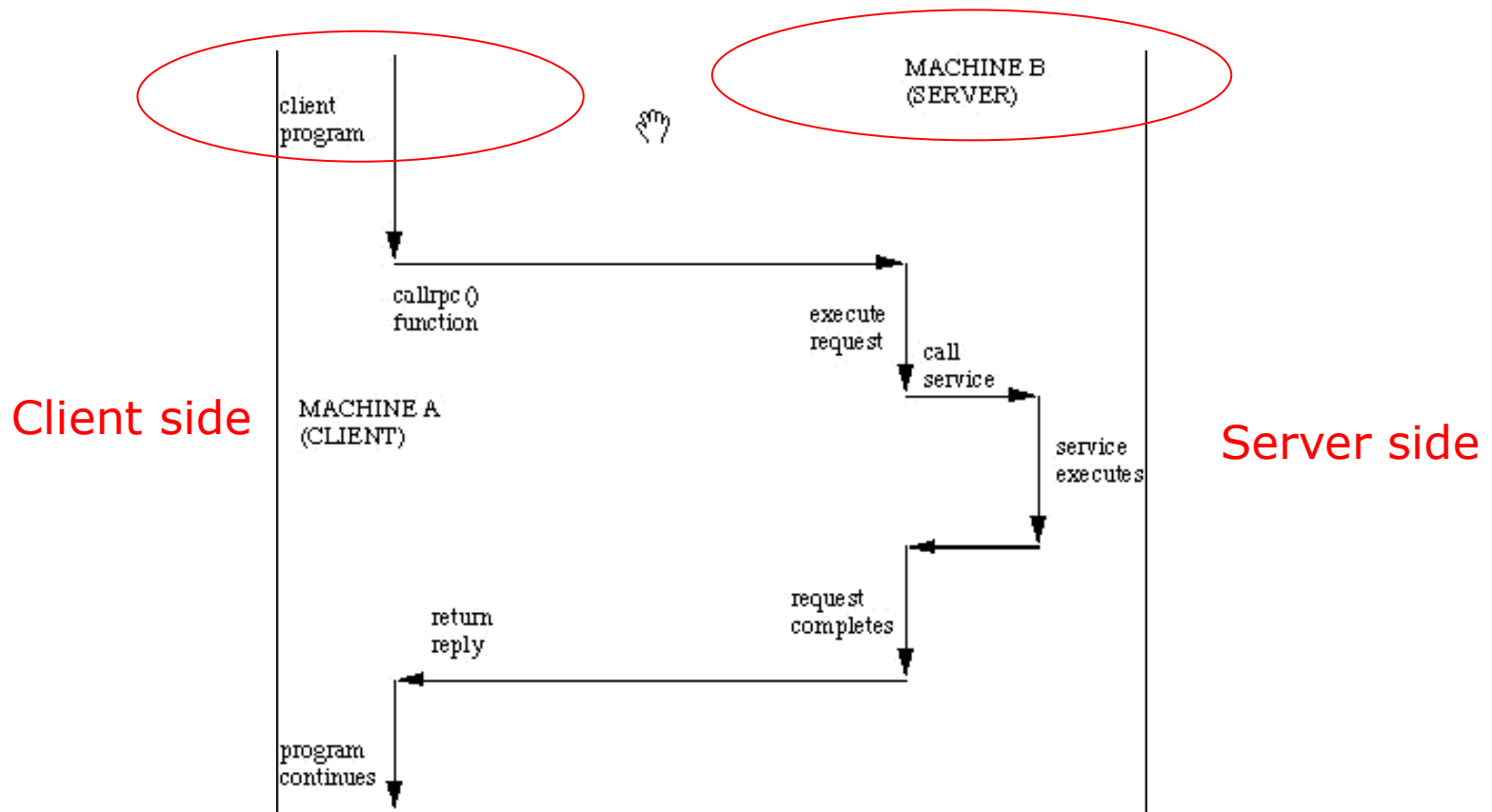
Operation and method remote invocation

- RPC – Remote Procedure Call
- RMI – Remote Method Invocation
- CORBA – Common Object Request Broker Architecture

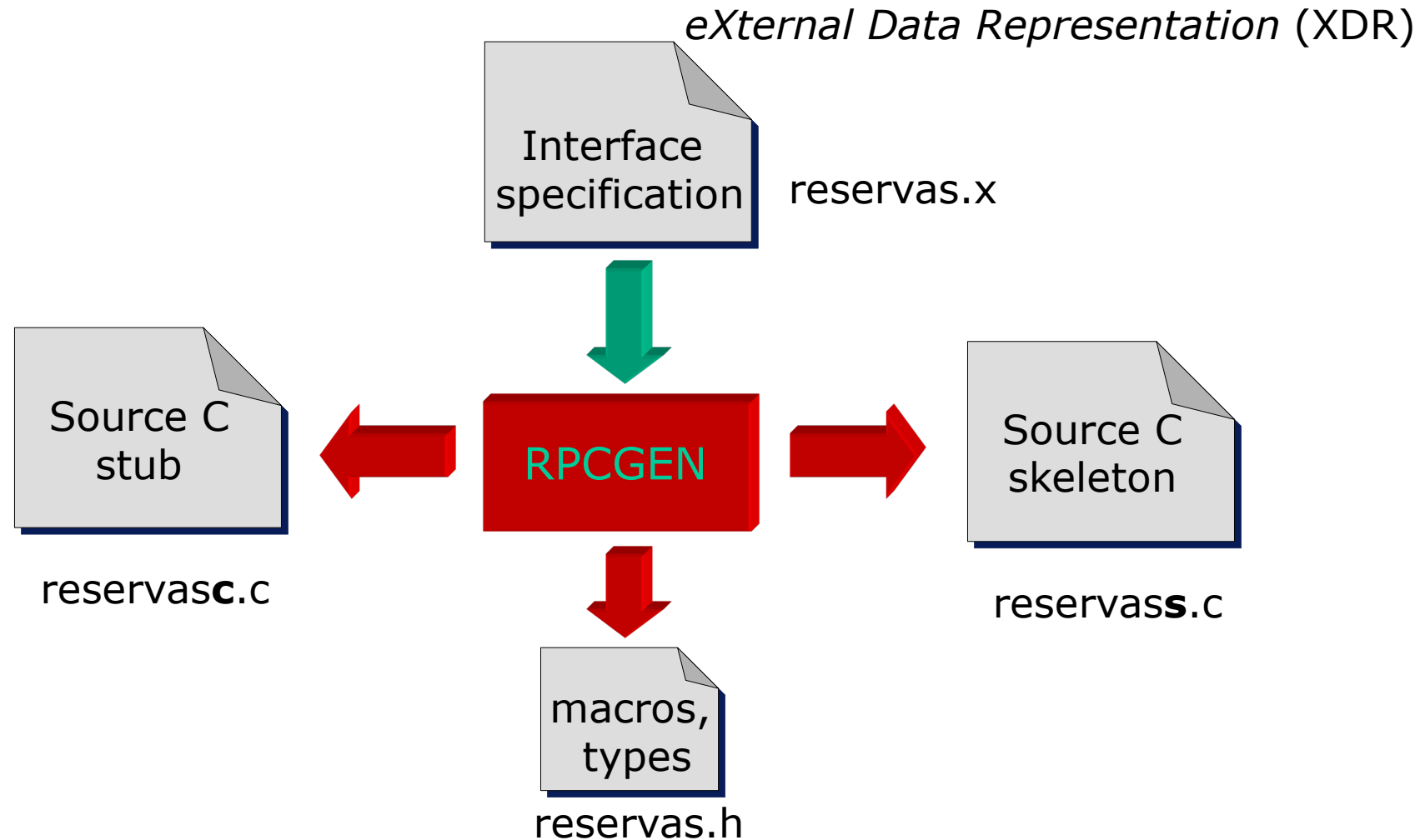
RPC

- Remote Procedure Call
 - Procedure call made remotely
 - Definition of remote procedure data types and interfaces
 - C Language – linux, unix based systems

RPC - Architecture



Generating stubs & skeletons in RPC



Reference: <http://www.cs.cf.ac.uk/Dave/C/node33.html>

XDR specification example

```
const MAX=10;  
typedef int Cantidad;  
typedef int Coste;  
typedef int Bool;
```

Constants and types similar to C

```
struct ReserveArgs{  
    char vuelo[MAX];  
    Cantidad reserva;  
};
```

RPC only admits 1 argument

XDR specification example

```
struct Resultado{  
    Bool exito;  
    Coste precio;  
};
```

RPC only has one result

```
program RESERVASVUELOS{  
    version VERSION {  
        Resultado RESERVA(ReserveArgs)=1;  
    }=3;  
}=6786;
```

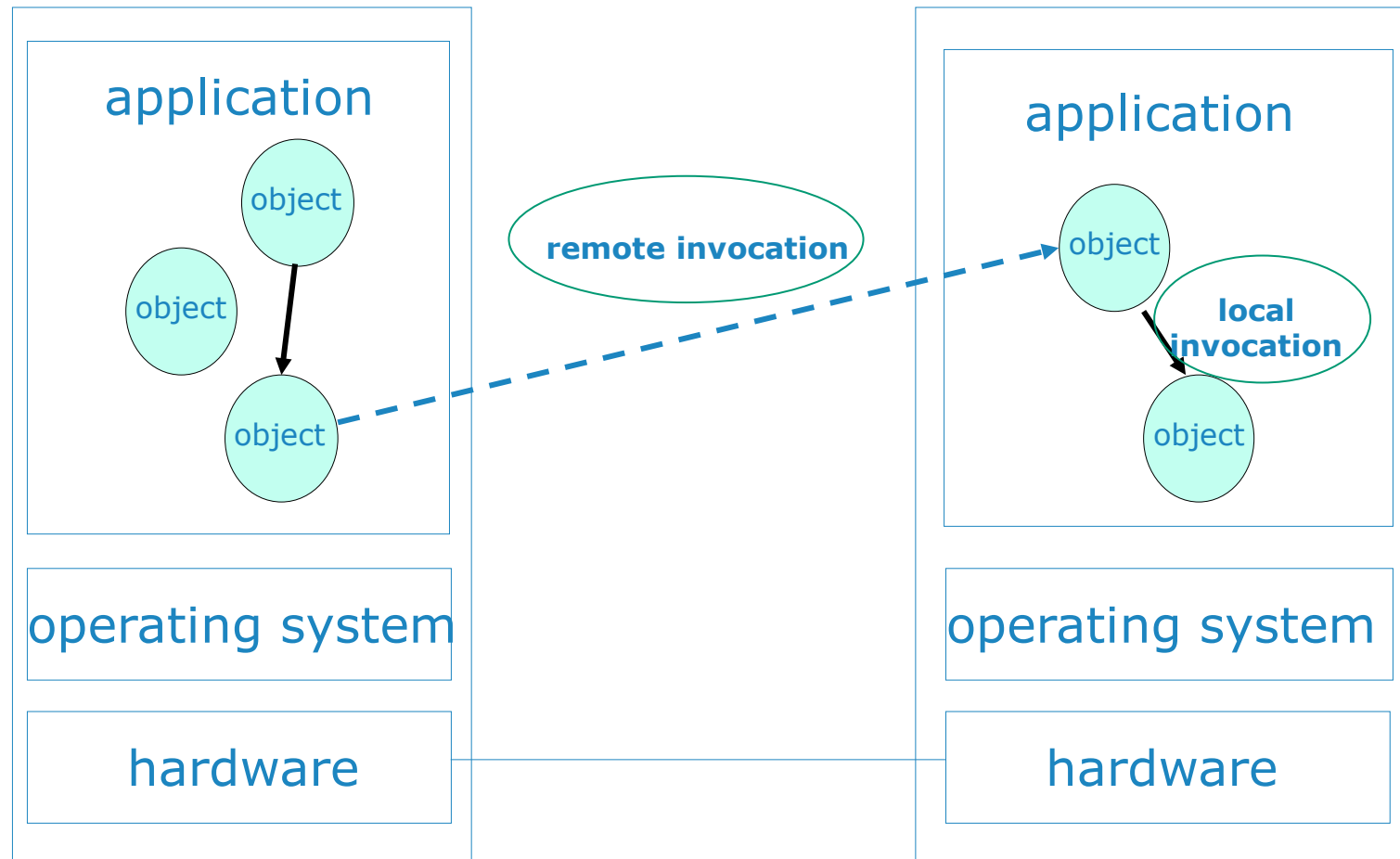
RPC gives a number to:

- program (1)
- version (3)
- procedure (6786)

RMI

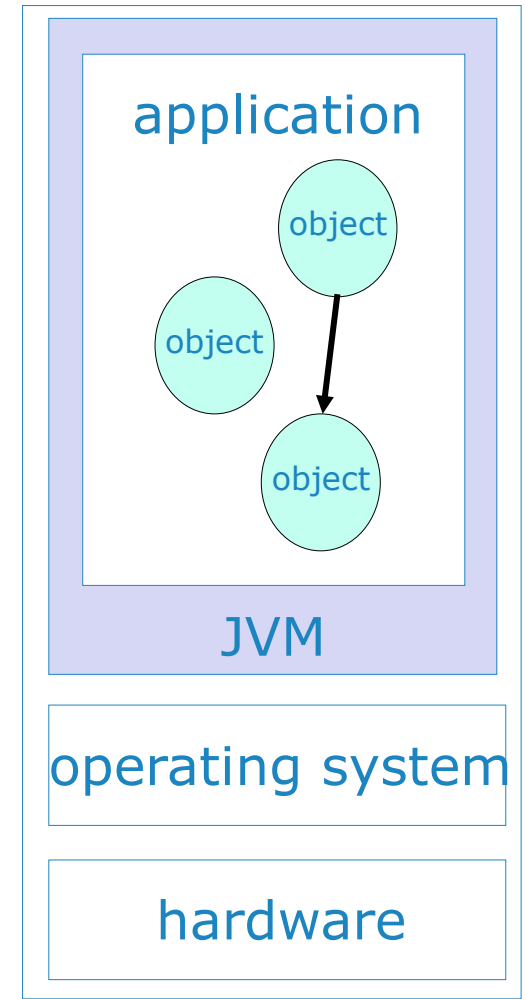
- Remote Method Invocation
 - Similar to RPC
 - Implemented in Java
 - Object Oriented
 - Remote object method invocation

Distributed Object Applications

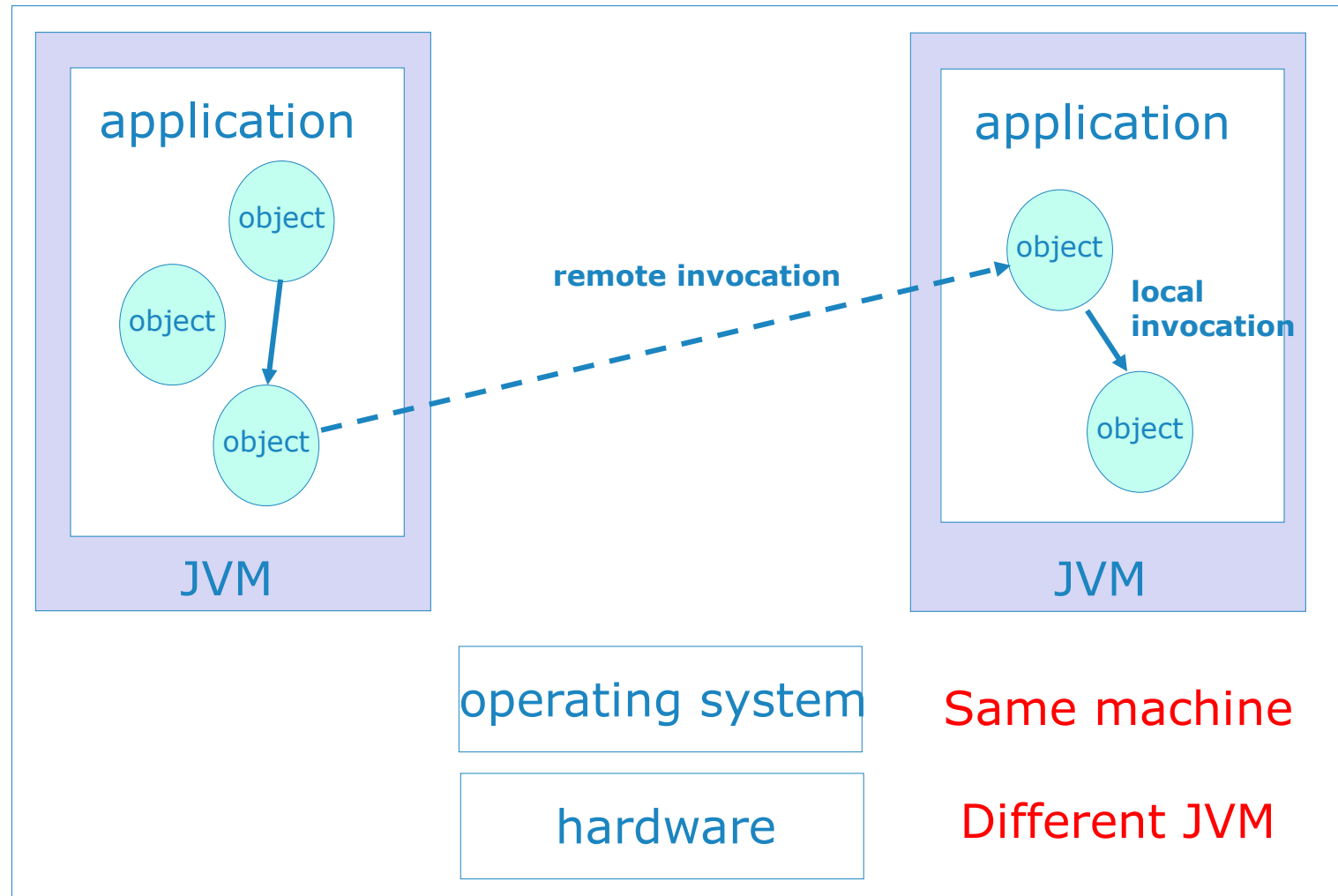


Local Java Application

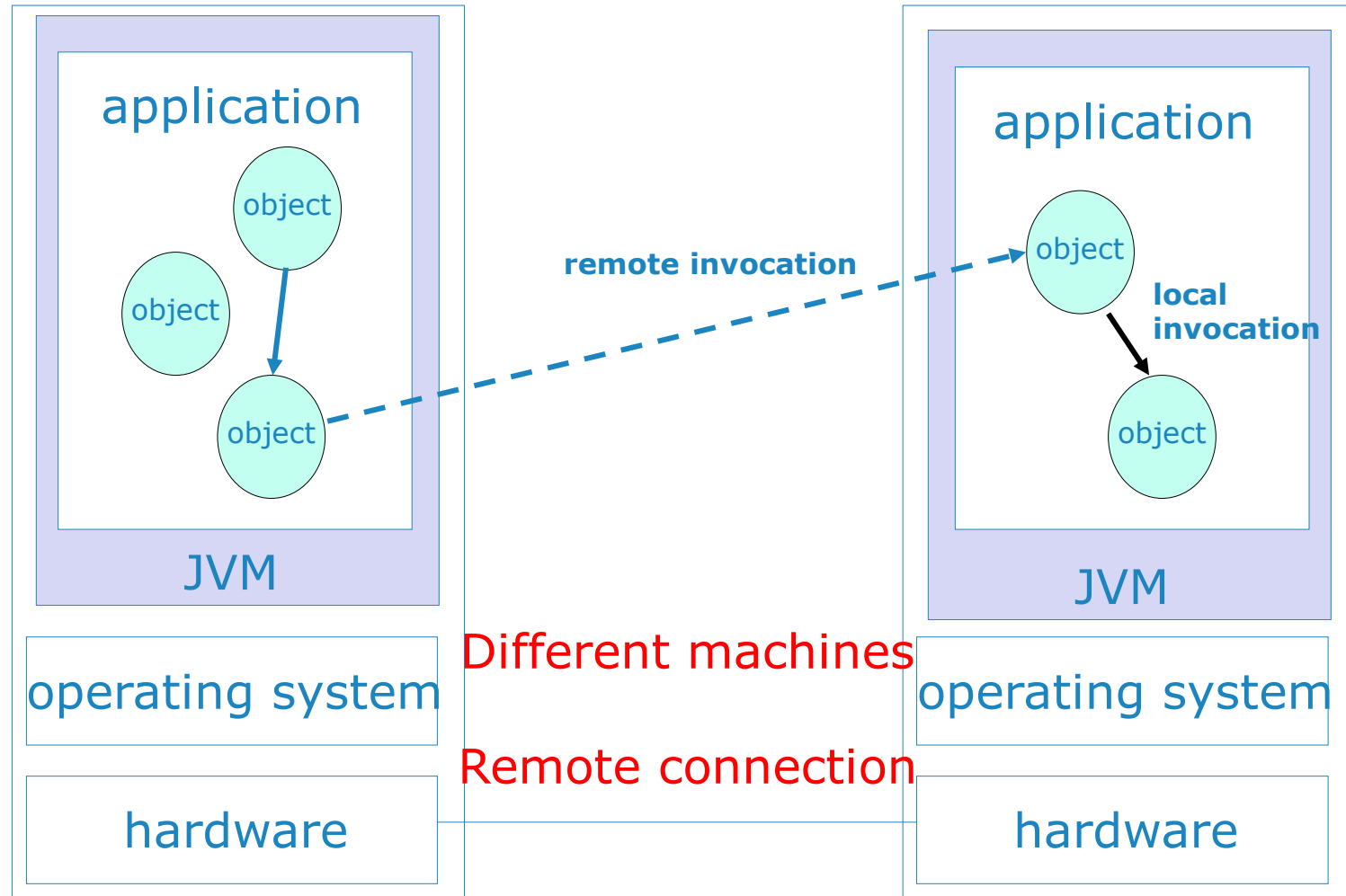
- Objects in the same JVM
- Local references to objects



Remote Java Application



Remote Java Application



Remote Java Application

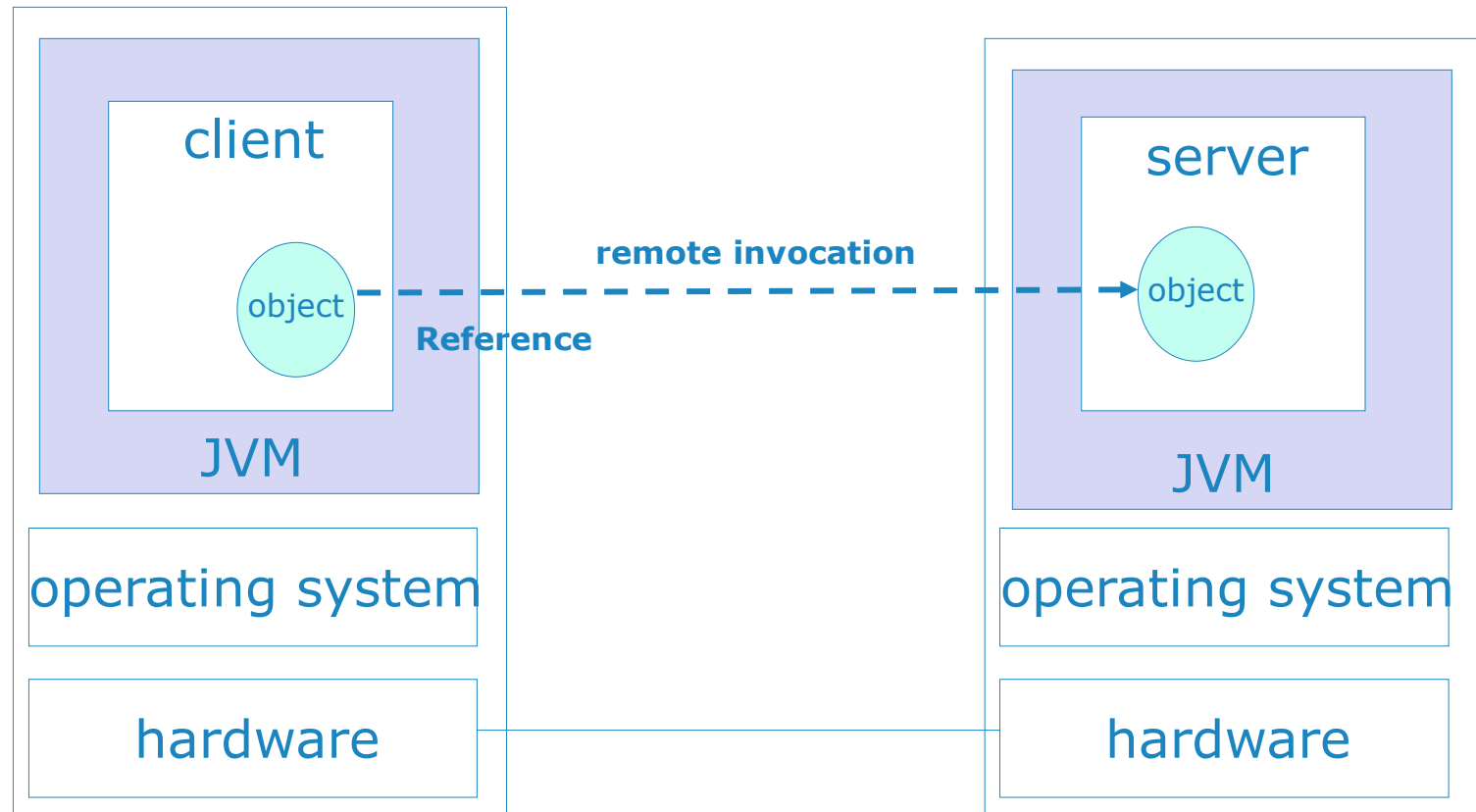
- Objects are located in different JVM (and possibly different machines)
- How they can be referenced?

Passing parameters in RMI

- Basic type
 - By value
- Object implementing *Remote* interface
 - References object like a local one
- Object implementing *Serializable* interface
 - It is transformed into bytes and sent

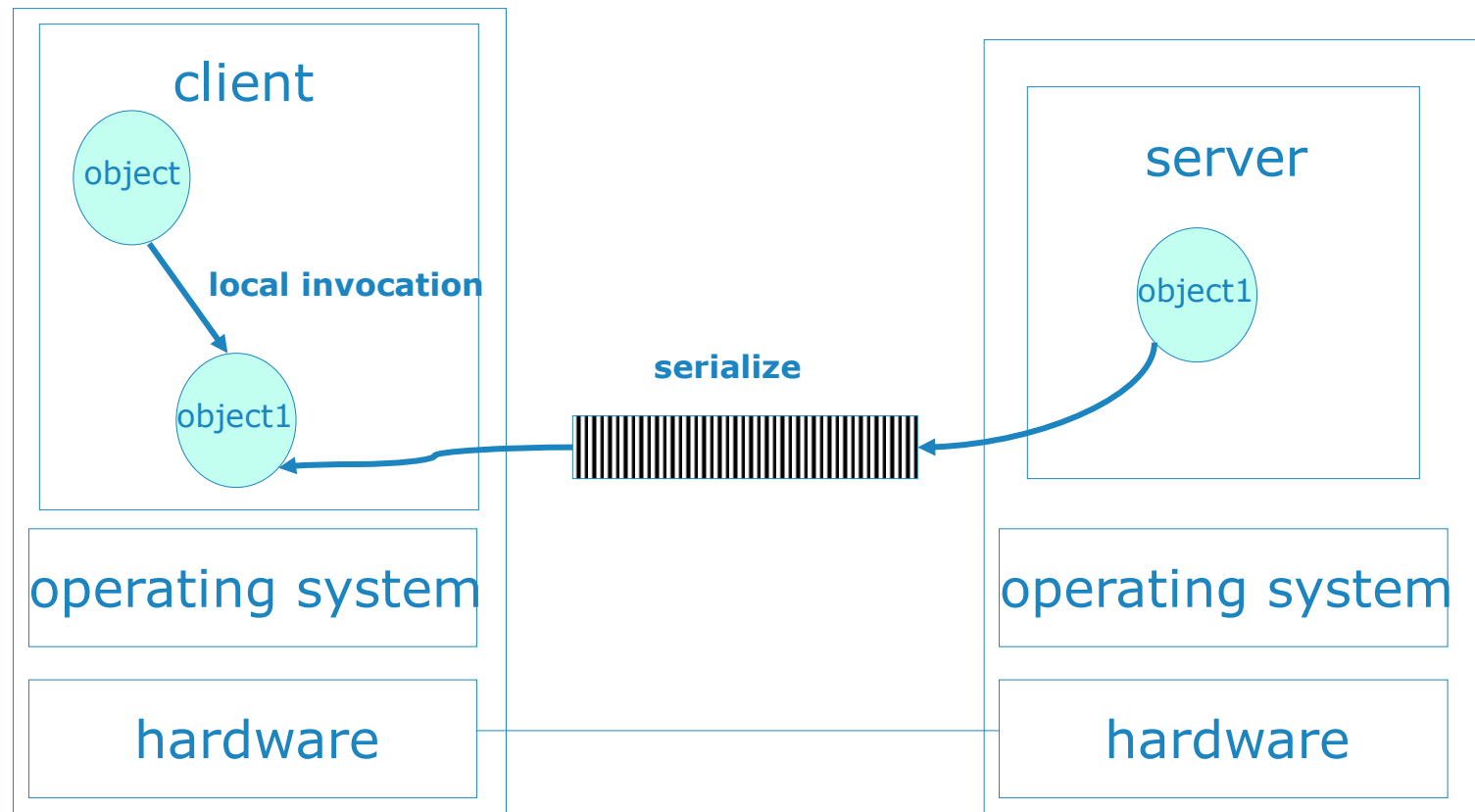
Remote

- Executing in server, invoked from client
- Object does not move from the JVM



Serializable

- Object is sent to the client (serialization)
- Local execution in client



Specific object problems

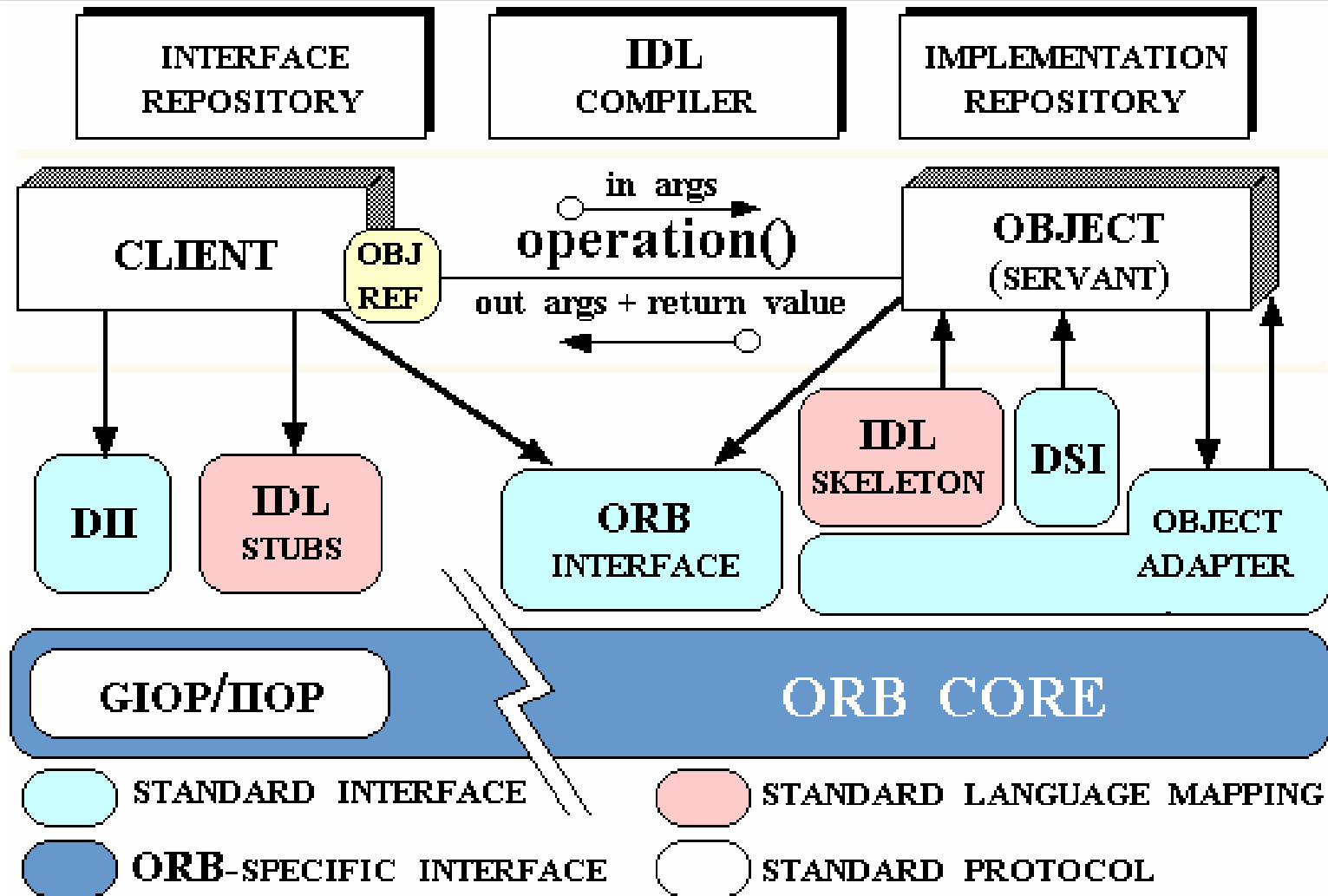
- Reference remote objects
- Create remote objects
- Memory collection (*garbage collection*)
- Exception management

CORBA

- Common Object Request Broker Architecture (CORBA)
 - Standard defined by Object Management Group (OMG) in 1995
 - Defines an architecture that allows connecting objects written in different programming languages
 - Communication between components is done in binary format
 - *Example of use: Client C++ connects to Java server*

Reference: <http://www.dre.vanderbilt.edu/~schmidt/corba-overview.html>

CORBA – Architecture and components



Sockets

- Sockets (XC)
 - Mechanism for sending remote information at TCP and UDP level
 - Two socket types
 - Stream for TCP
 - Datagram for UDP
 - Lower level where communication can be done.
 - It is needed to define the protocol, request and response format, etc.
 - Operations to
 - open / close connection
 - read / write in the socket

HTTP

- HyperText Transfer Protocol
- Until version 2 works over TCP (stream sockets)
- Application level protocol for distributed systems
- Supports different types of information

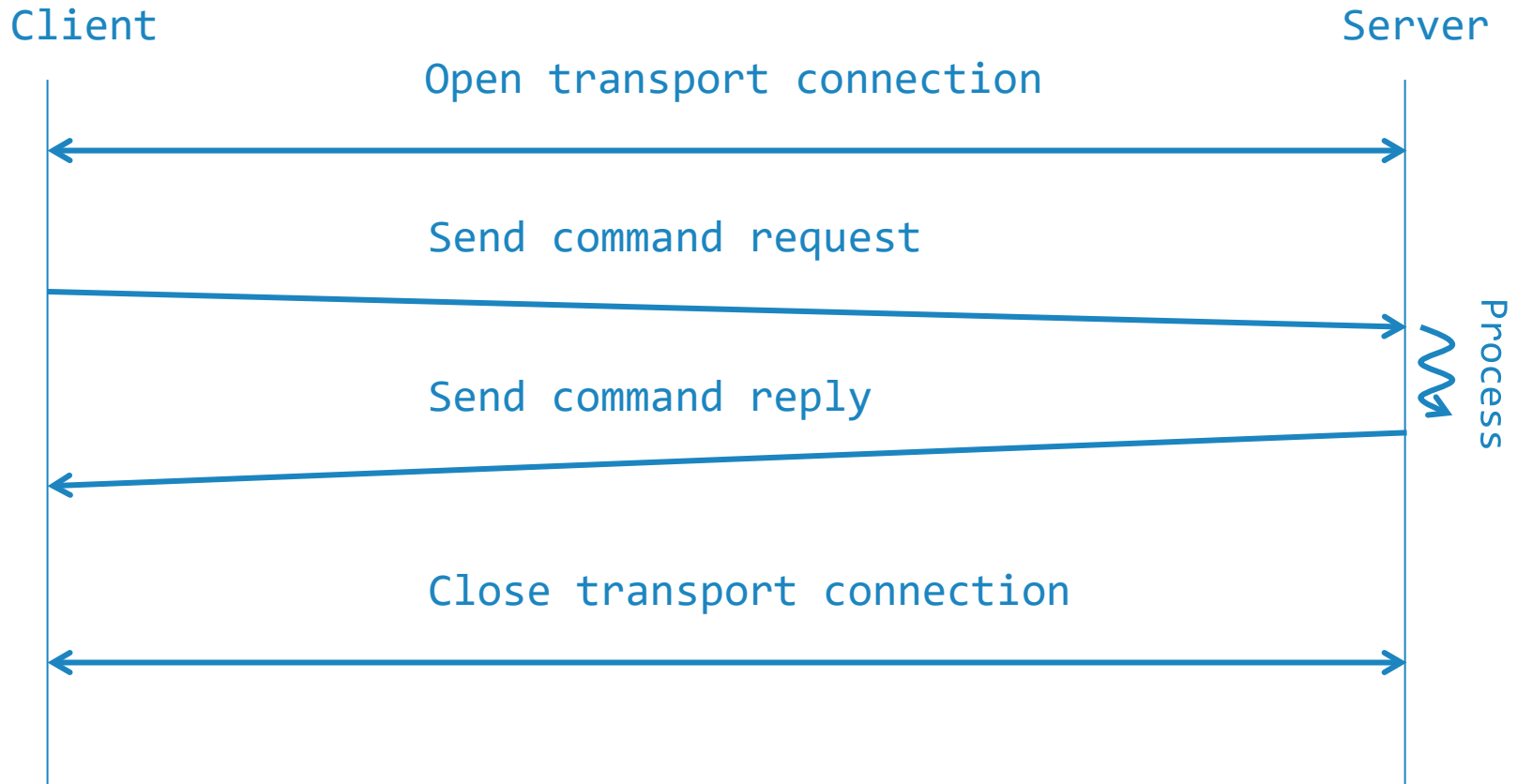
- Versions: 1.1 (rfc9112, June 2022), 2 (rfc9113, June 2022), 3 (rfc9114, June 2022)
 - <https://www.rfc-editor.org/rfc/rfc9110>
 - https://en.wikipedia.org/wiki/Comparison_of_web_browsers#Protocol_support

Reference: <https://w3techs.com/technologies/details/ce-http3>

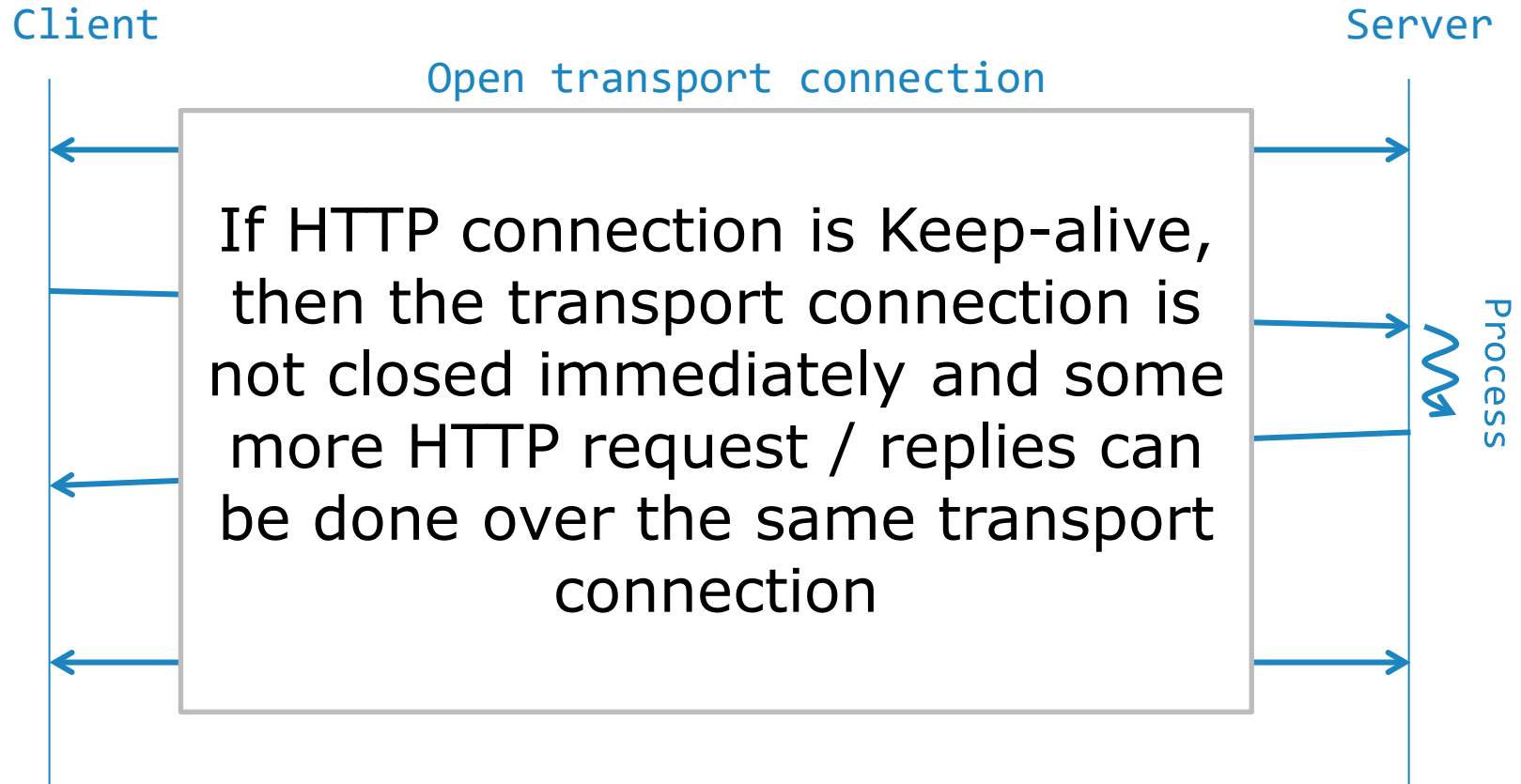
HTTP features

- Stateless protocol
- Request / Reply
- External mechanisms for state control
 - Cookies
 - Sessions
- Mostly works over TCP
- Port 80 by default

HTTP basic description



HTTP basic description

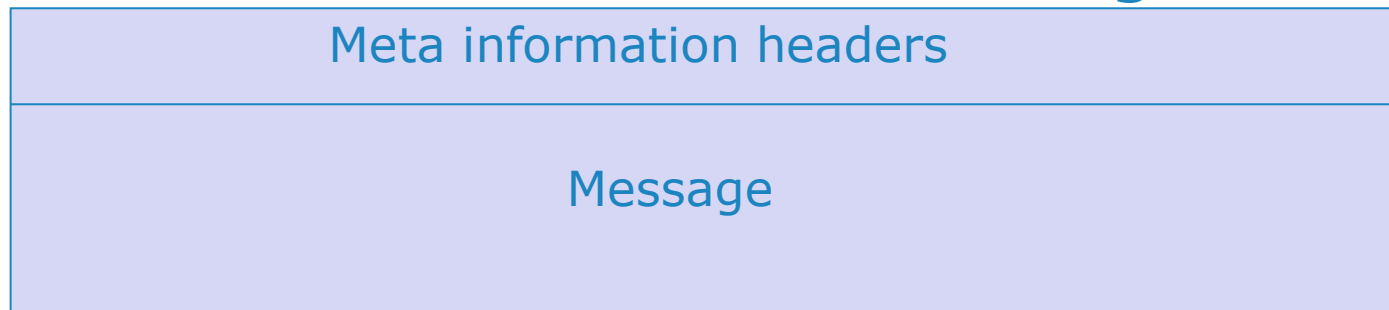


HTTP evolution

- 0.9 version: Byte strips without format



- 1.0 version: Formatted messages

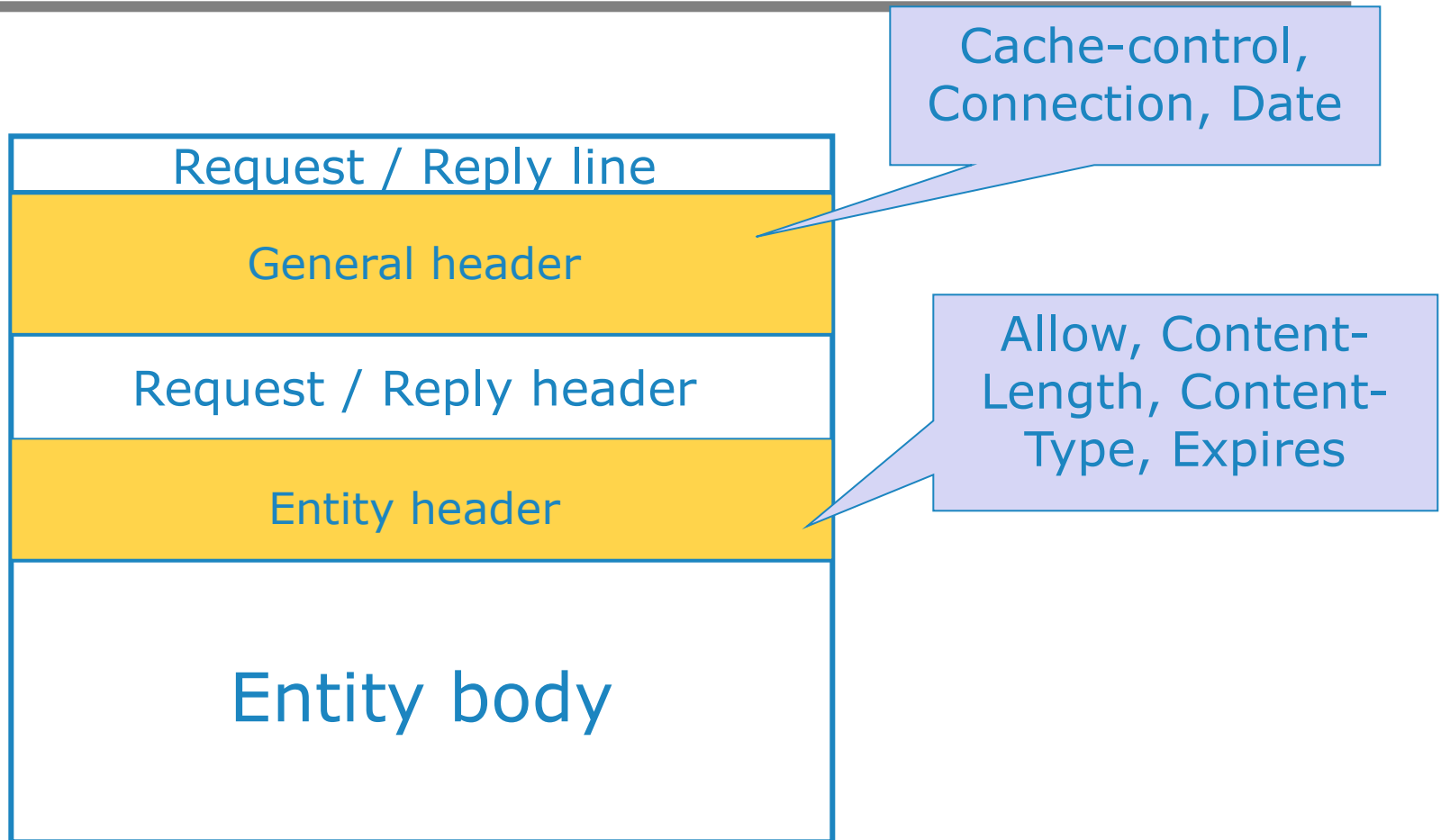


- 1.1 version: New features (Connection persistence, proxies, data cache, etc.)
 - Formatted messages

HTTP evolution

- Version 2: Common semantics, change in data sending
 - Binary, multiplexing
- Version 3: HTTP over QUIC protocol, more details in next topics
 - UDP, use of some HTTP/2 features

HTTP messages



HTTP messages: Request

GET, POST,
HEAD, PUT,
DELETE, COPY

HTTP_method SP Resource_URL SP HTTP_version CRLF

General header

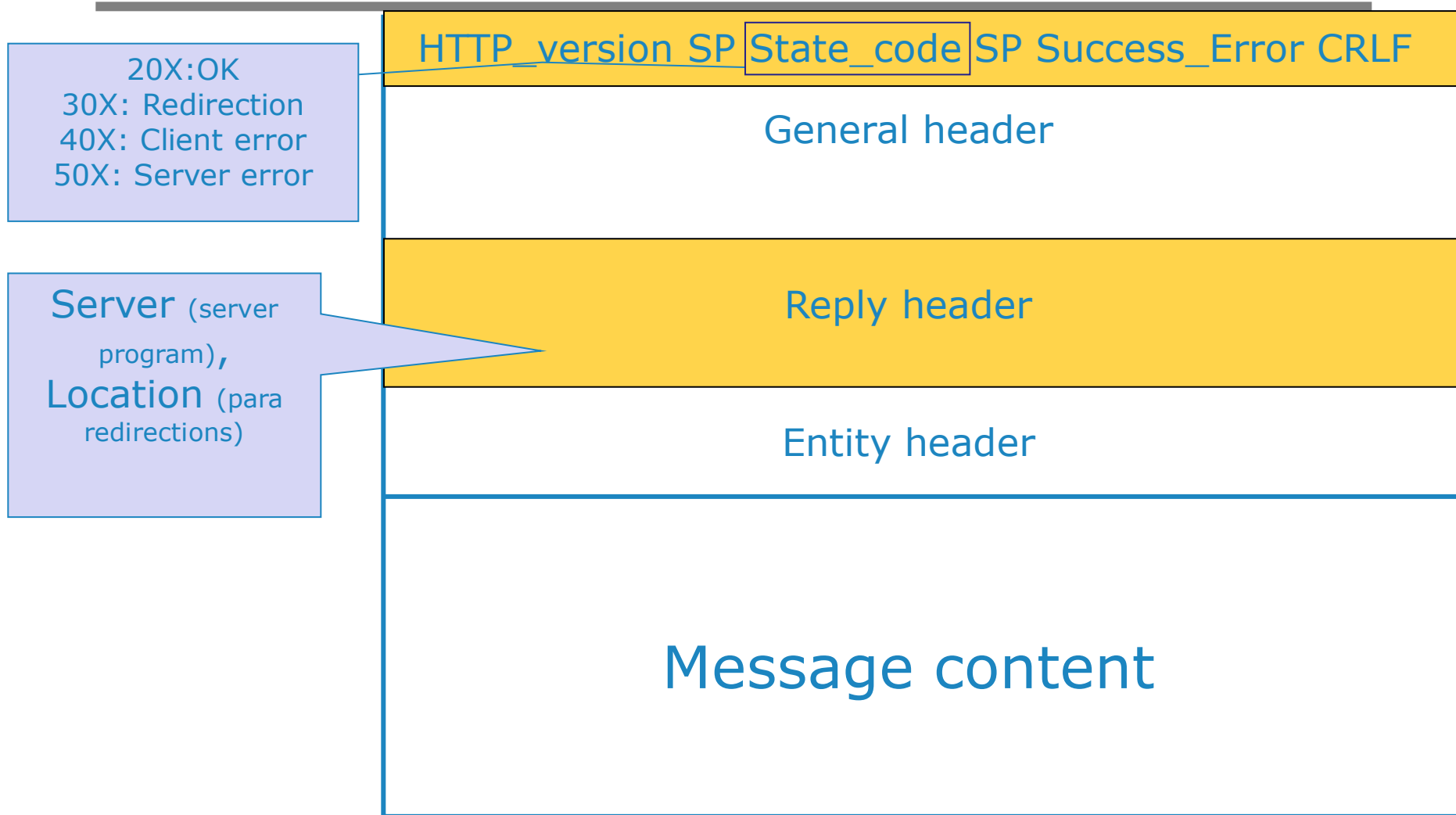
Accept
(supported
contents), Host
(server name),
User-Agent
(browser)

Request header

Entity header

Message content

HTTP message: Reply



Reference: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers>

HTTP methods (“No modification”)

- **GET.** Requests the specified resource. Should only retrieve data. No other effect. May pass parameters.
- **HEAD.** Response identical to GET without the body (only headers).
- **TRACE.** Echoes back the received request.
- **OPTIONS.** Returns the HTTP methods that the server supports for the specified URL.

HTTP methods (“Modification”)

- **POST.** Submits data to be processed → update, creation.
Examples: HTML form, annotation, message, item to add to a database, ...
- **PUT.** Uploads the specified resource (creation).
- **PATCH.** Applies partial modifications to the resource (update).
- **DELETE.** Deletes the specified resource.

HTTP GET Request example

GET /search?q=myBook HTTP/1.1

Host: www.google.com

User-Agent: Mozilla/5.0 ...

MIME types

Accept: text/xml,application/xml,text/html,
text/plain,image/png, ...

Accept-Language: da,en-us, ...

Accept-Encoding: gzip,deflate

Accept-Charset: ISO-8859-1,utf-8 ...

Keep-Alive: 300 Time out (in s.)

Connection: keep-alive Persistence

Referer: http://www.google.com/

HTTP GET Response example

HTTP/1.1 200 OK

Date: Fri, 17 Sep 2021 07:59:01 GMT

Server: Apache/2.0.50 (Unix) ...

Last-Modified: Tue, 24 Feb 2021 08:32:26 GMT

Accept-Ranges: bytes

Accept range req.

Content-Length: 2810

Content-Type: text/html

... body content ...

HTTP 2 – Some concepts

- Based on SPDY (speedy)
- Methods, state codes and semantics are the same as HTTP/1.1
- What changes?
 - The way of sending data between client and server
 - Appears the Frame concept
 - Data is sent in binary format, not textual as it was until now

HTTP 2 – Some concepts

- Performance improvement is pursued:
 - Perception of latency by the end user
 - Usage of network and server resources
 - Do not send repetitive information
 - Multiplex several requests to a web server over a single TCP connection
- Allows the server to send responses to the client (Server push) without the need for a prior request
 - Advance information sending
 - In HTTP 1.1 requests always come from the client

HTTP 2 - Summary

1. One TCP connection

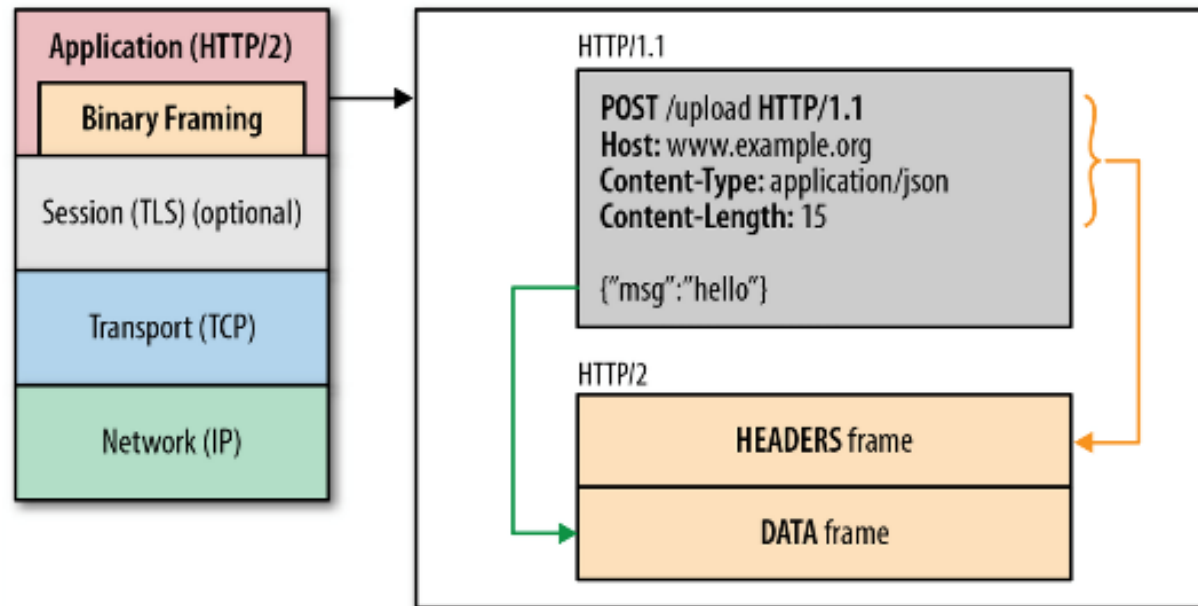
2. Request → Stream

- Streams are multiplexed
- Streams are prioritized

3. Binary framing layer

- Prioritization
- Flow control
- Server push

4. Header compression (HPACK)



Reference: <https://developers.google.com/web/fundamentals/performance/http2?hl=es>

HTTP 2 – Some references

- RFC HTTP/2
 - <https://tools.ietf.org/html/rfc9113>
- Articles about HTTP/2
 - <http://resources.infosecinstitute.com/http2-faster-and-safer-web-enforcing-strong-encryption-as-the-de-facto-standard/#gref>
 - <https://hpbn.co/http2/>
 - <https://www.smashingmagazine.com/2017/04/guide-http2-server-push/>
 - <https://raiolanetworks.com/blog/http2-http3/>
 - <https://caniuse.com/?search=http2>

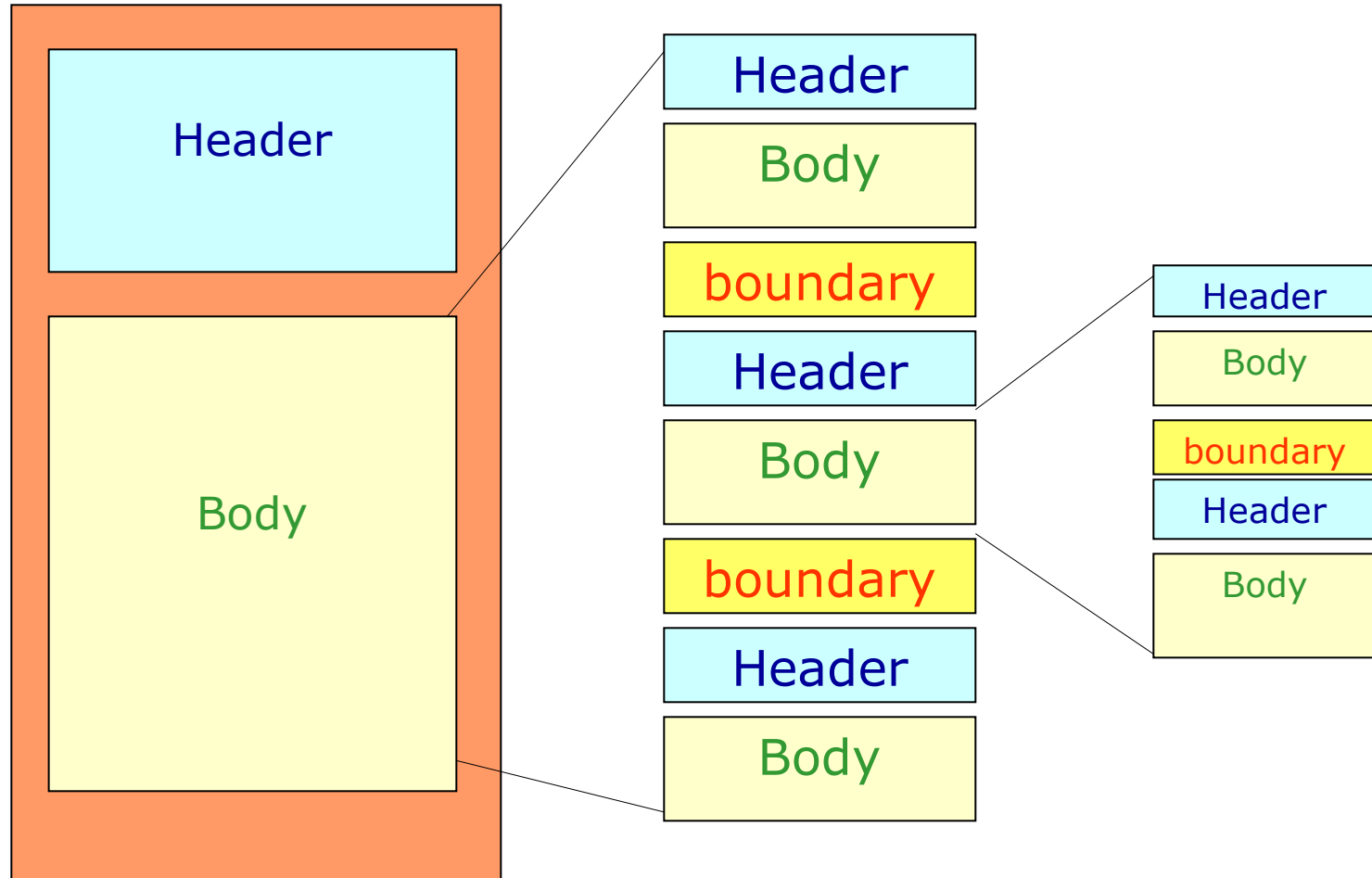
MIME, HTML, HTML5

- Information formats used in e-mail and web
- They allow including / linking different types of information
 - Examples:
 - Images or documents inside the MIME message.
 - Web pages linking images, documents, videos, etc.
- RFC 1521 - MIME
 - <https://datatracker.ietf.org/doc/html/rfc1521>

MIME

- MIME (Multipurpose Internet **Mail** Extensions)
 - Allows sending contents for multiple formats
- Its use has been extended to other application contexts: Web, S.O.,
- Generalization of e-mail
 - Messages usage for sending different contents
 - Normalize coding of non-ASCII

MIME format



MIME content types

- Content-Type field
 - It has two parts: type / subtype
- Types:
 - application, audio, image, message, multipart, text, video
- Examples:
 - image/gif, image/jpeg, image/png, ...
 - text/plain, text/html, message/rfc822,
 - application/postscript, application/msword, ...

Content types examples

- multipart/mixed

From: John Doe <example@example.com>

MIME-Version: 1.0

Content-Type: multipart/mixed; boundary="XXXXboundary text"

This is a multipart message in MIME format.

--XXXXboundary text

Content-Type: text/plain

this is the body text

--XXXXboundary text

Content-Type: text/plain;

Content-Disposition: attachment; filename="test.txt"

this is the attachment text

--XXXXboundary text--

Content types examples

- multipart/mixed

From: John Doe <example@example.com>
MIME-Version: 1.0

Mail header
Who sends, MIME version

Content-Type: multipart/mixed; boundary="XXXXboundary text"

This is a multipart message in MIME format.

Organization of content
inside the message

--XXXXboundary text
Content-Type: text/plain
this is the body text

First part, text type inside the message

--XXXXboundary text
Content-Type: text/plain;
Content-Disposition: attachment; filename="test.txt"
this is the attachment text

Second part, attached file

--XXXXboundary text--
End of message

From: "Senders Name" <sender@sendersdomain.com>
To: "Recipient Name" <somerecipient@recipientdomain.com>
Message-ID: <5bec11c119194c14999e592feb46e3cf@sendersdomain.com>
Date: Sat, 24 Sep 2020 15:06:49 -0400
Subject: Sample Multi-Part
MIME-Version: 1.0
Content-Type: multipart/alternative;
boundary="-----=_NextPart_DC7E1BB5_1105_4DB3_BAE3_2A6208EB099D"

-----=_NextPart_DC7E1BB5_1105_4DB3_BAE3_2A6208EB099D
Content-type: text/plain; charset=iso-8859-1
Content-Transfer-Encoding: quoted-printable

Sample Text Content

-----=_NextPart_DC7E1BB5_1105_4DB3_BAE3_2A6208EB099D
Content-type: text/html; charset=iso-8859-1
Content-Transfer-Encoding: quoted-printable

<html>
<head>
</head>
<body> <div style=3D"FONT-SIZE: 10pt; FONT-FAMILY: Arial">Sample HTML = Content</div></body>
</html>

-----=_NextPart_DC7E1BB5_1105_4DB3_BAE3_2A6208EB099D--

From: "Senders Name" <sender@sendersdomain.com> **Mail headers**

To: "Recipient Name" <somerecipient@recipientdomain.com>

Message-ID: <5bec11c119194c14999e592feb46e3cf@sendersdomain.com>

Date: Sat, 24 Sep 2020 15:06:49 -0400

Subject: Sample Multi-Part

MIME-Version: 1.0

**Message type and
separation**

Content-Type: multipart/alternative;
boundary="-----=_NextPart_DC7E1BB5_1105_4DB3_BAE3_2A6208EB099D"

-----=_NextPart_DC7E1BB5_1105_4DB3_BAE3_2A6208EB099D

Content-type: text/plain; charset=iso-8859-1

Content-Transfer-Encoding: quoted-printable

First element, text

Sample Text Content

-----=_NextPart_DC7E1BB5_1105_4DB3_BAE3_2A6208EB099D

Content-type: text/html; charset=iso-8859-1

Content-Transfer-Encoding: quoted-printable

Second element, html

<html>

<head>

</head>

<body> <div style=3D"FONT-SIZE: 10pt; FONT-FAMILY: Arial">Sample HTML = Content</div></body>

</html>

-----=_NextPart_DC7E1BB5_1105_4DB3_BAE3_2A6208EB099D--

From: "Senders Name" <sender@sendersdomain.com> **Mail headers**

To: "Recipient Name" <somerecipient@recipientdomain.com>

Message-ID: <5bec11c119194c14999e592feb46e3cf@sendersdomain.com>

Date: Sat, 24 Sep 2020 15:06:49 -0400

Subject: Sample Multi-Part

MIME-Version: 1.0

**Message type and
separation**

Content-Type: multipart/alternative;
boundary="-----=_NextPart_DC7E1BB5_1105_4DB3_BAE3_2A6208EB099D"

-----=_NextPart_DC7E1BB5_1105_4DB3_BAE3_2A6208EB099D

Content-type: text/plain; charset=iso-8859-1

Content-Transfer-Encoding: quoted-printable

First element, text

Sample Text Content

-----=_NextPart_DC7E1BB5_1105_4DB3_BAE3_2A6208EB099D

Content-type: text/html; charset=iso-8859-1

Content-Transfer-Encoding: quoted-printable

Second element, html

<html>

<head>

</head>

<body> <div style=3D"FONT-SIZE: 10pt; FONT-FAMILY: Arial">Sample HTML = Content</div></body>

</html>

-----=_NextPart_DC7E1BB5_1105_4DB3_BAE3_2A6208EB099D--

Content types examples

- multipart/form-data: HTML Forms sending format

- boundary

- Content-Type: multipart/form-data, boundary=AaB03x

- Content-Disposition

- AaB03x

- Content-Disposition: form-data; name="file";
filename="file1.html"

- Content-Type: text/html

- ... Contents of the file file1.html ...

- AaB03x--

Some MIME headers

- MIME-Version
- Content-Type
- Content-Transfer-Encoding
- Content-ID
- Content-Disposition
- Content-Description
- Content-Language

- Some more examples:
 - <https://www.oreilly.com/openbook/mh/mulmes.htm>

HTML

- Defined by W3C for web pages creation
- Based on tags: `<tag> ... </tag>` or `<tag/>`
- Define the document logical structure
 - Not the display, as browser decides
- More structured and complex
- Exercise
 - How HTML tags are used?
 - And attributes?

HTML

- Response

- Tags

- Content is delimited by a start and end tag
 - `<tag> Content </tag>`

- Attributes

- They add semantics to a tag
 - `<tag atrib = "value"> Content </tag>`

HTML

- HTML document structure
 - `<html>` `</html>` Start / End of document
 - `<head>` `</head>` Document header elements, not shown in the browser. Include things like the title or metadata about the page (request time, if it can be cached, etc.)
 - `<body>` `</body>` HTML document body, what the browser shows

Reference: <http://www.w3schools.com/html/default.asp>

HTML

- Javascript
 - Initially, add scripting capabilities in the client (browser)
 - Validate data in browser
 - Minimize server connections
 - Local alert windows
 - More complex and complete, it allows moving part of the web application to the client side
 - Now you can also have javascript in the server side
 - Examples of javascript usage: node (server), angular (client), vue (client), etc.

References: <https://nodejs.org/en/> <https://angular.dev/> <https://vuejs.org/>

HTML

- Cascade Style Sheets (CSS)
 - Style sheets
 - They are used to format HTML elements
 - Allows the clear separation between the document structure and its graphical style
 - The information is applied on cascade, hence its name

Reference: <http://www.w3schools.com/css/default.asp>

HTML5

- New HTML version, defined by W3C and Web Hypertext Application Technology Working Group (WHATWG) -
<https://html.spec.whatwg.org/>
- Still under construction (it is not finished yet)
- Browsers support some of its new features
- Basic rules
 - Based in HTML, CSS, DOM, Javascript
 - Reduce external plug-ins need
 - Improve error management
 - More markup to reduce scripting
 - Device independence

HTML5

- New HTML version, defined by W3C and Web Hypertext Application Technology Working Group (WHATWG) - <https://html.spec.whatwg.org/>

- Still under development (not yet) released
- Browser support
- Basic rules
 - Based on previous versions
 - Reduced complexity
 - Improved performance
 - More markup to reduce scripting
 - Device independence

HTML

Living Standard

HTML5

- Some of the new elements:
 - `<canvas>` to draw 2D elements
 - `<video>` and `<audio>` to manage multimedia files
 - Local storage support
 - Specific content elements, like `<article>`, `<footer>`, `<header>`, `<nav>`, `<section>`
 - New built-in form controls, like calendar, date, time, email, URL, search (depend on browser support)

HTML5

- Video element in action

```
<video width="320" height="240"  
controls="controls">
```

```
  <source src="movie.mp4" type="video/mp4" />
```

```
  <source src="movie.ogv" type="video/ogg" />
```

```
  Your browser does not support the video tag.
```

```
</video>
```

HTML5

- References:
 - <https://html.spec.whatwg.org/multipage/>

XML

- XML: eXtensible Markup Language
- Designed to transport and store data
 - HTML used to display data
 - HW & SW-independent
- XML
 - To carry data. To process it automatically.
 - **Users** must define their own **tags**
 - **Users**: Private users and SDO (Standards Developing Organizations)

XML structure & syntax

- XML:

- Tree structure.
- Elements, attributes & text.

- Example:

```
<book category="COOKING">  
  <title lang="en">Everyday Italian</title>  
  <author>Giada De Laurentiis</author>  
  ...  
</book>
```

XML structure & syntax

- XML:

- Tree structure

- Elements, attributes & text.

- Example:

`<book category="COOKING">`

`<title lang="en">Everyday Italian</title>`

`<author>Giada De Laurentiis</author>`

...

`</book>`

XML structure & syntax

- First line (example):
`<?xml version="1.0" encoding="ISO-8859-1"?>`
- XML simple syntax:
 - Closing tag mandatory
 - Tags are case sensitive
 - Elements could be nested:
`<a> <b>...</b> <c>...</c> </a>`
(a parent, b, c children , b, c siblings)
 - Root element needed (It is unique and defines the document type)

XML structure & syntax

- XML simple syntax (cont.):

- ...

- Attribute values must be quoted

- Entity references

- Examples:

- predefined entity

- "

- &

- represented character

- "

- &

- Comments:

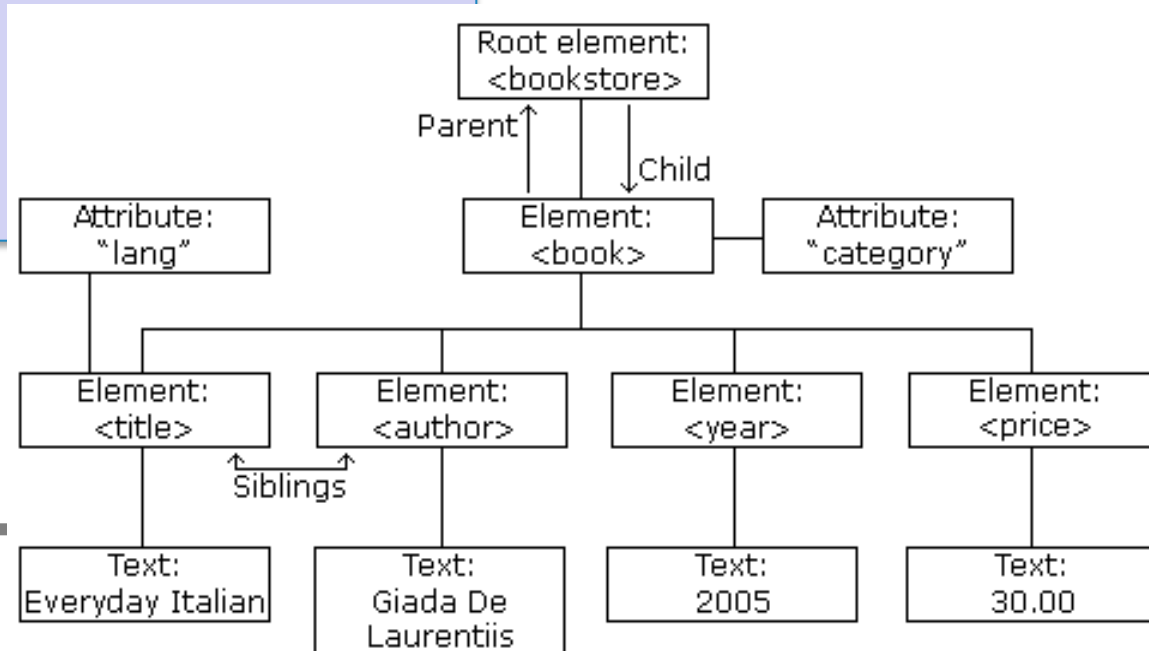
- <!-- ... -->

XML - other issues

- Attributes vs. Elements: Design decision
- Name conflicts:
 - Namespaces: Differentiate element names defined in different places (possibly by application or standard)
 - Defined with xmlns attribute
 - xmlns:prefix = "URI"
 - URIs are an easy way to define **unique** namespaces
- How to define tags and structure? Using schemas
 - Some examples ...

XML simple example

```
<bookstore>
  <book category="COOKING">
    <title lang="en">Everyday Italian</title>
    <author>Giada De Laurentiis</author>
    <year>2005</year>
    <price>30.00</price>
  </book>
  <book category="CHILDREN">
    <title lang="en">Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
    <price>29.99</price>
  </book>
  ...
</bookstore>
```



XML: Idea of Schema

- XML Schema Definition, **XSD**
- Content of the file “**note.xsd**”:

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="note">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="to" type="xs:string"/>
        <xs:element name="from" type="xs:string"/>
        <xs:element name="heading" type="xs:string"/>
        <xs:element name="body" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

<http://www.w3schools.com/xml/>

namespace where the **schema** is defined,
the namespace should be prefixed xs.

root element

complexType: contains other elements

sequence: child elements must
appear in the same order

- Reference to the XSD defined in “**note.xsd**”:

```
<?xml version="1.0"?>
<note xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="note.xsd">
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

XSD schema defined in
location “note.xsd”

XML: Idea of Schema

- XML Schema Definition, **XSD**
- Content of the file “**note.xsd**”:

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="note">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="to" type="xs:string"/>
        <xs:element name="from" type="xs:string"/>
        <xs:element name="heading" type="xs:string"/>
        <xs:element name="body" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

<http://www.w3schools.com/xml/>

namespace where the **schema** is defined,
the namespace should be prefixed **xs**

root element

complexType: contains other elements

sequence: child elements must
appear in the same order

- Reference to the XSD defined in “**note.xsd**”:

```
<?xml version="1.0"?>
<note xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="note.xsd">
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

XSD schema defined in
location “note.xsd”

XML: Idea of Schema

- XML Schema Definition, **XSD**
- Content of the file “**note.xsd**”:

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="note">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="to" type="xs:string"/>
        <xs:element name="from" type="xs:string"/>
        <xs:element name="heading" type="xs:string"/>
        <xs:element name="body" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

<http://www.w3schools.com/xml/>

namespace where the **schema** is defined,
the namespace should be prefixed (**xs**)

root element

complexType: contains other elements

sequence: child elements must
appear in the same order

- Reference to the XSD defined in “**note.xsd**”:

```
<?xml version="1.0"?>
<note xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="note.xsd">
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

XSD schema defined in
location “note.xsd”

XML: Idea of Schema

- XML Schema Definition, **XSD**
- Content of the file “**note.xsd**”:

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="note">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="to" type="xs:string"/>
        <xs:element name="from" type="xs:string"/>
        <xs:element name="heading" type="xs:string"/>
        <xs:element name="body" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

<http://www.w3schools.com/xml/>

namespace where the **schema** is defined,
the namespace should be prefixed xs.

root element

complexType: contains other elements

sequence: child elements must
appear in the same order

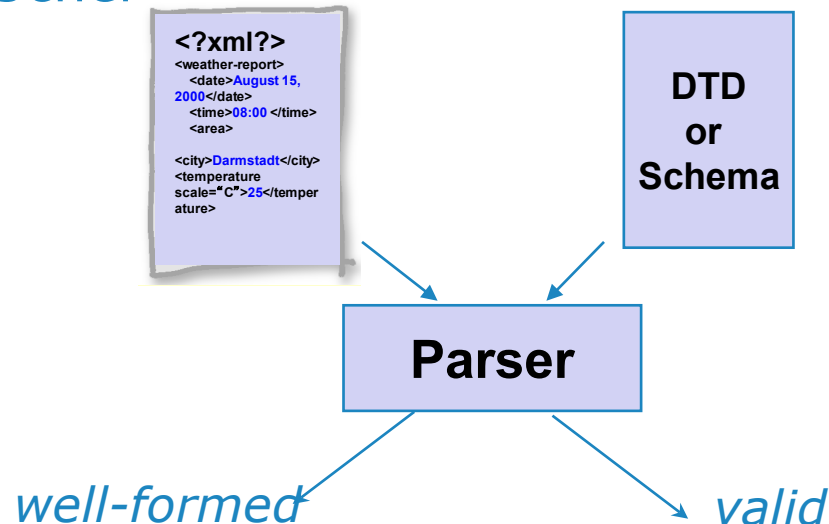
- Reference to the XSD defined in “**note.xsd**”:

```
<?xml version="1.0"?>
<note xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="note.xsd">
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

XSD schema defined in
location “note.xsd”

XML: validity

- "well-formed" XML document:
 - Document that satisfies **XML syntax rules**
- "valid" XML document :
 - Well-formed document that also conforms to **some rules** specified in a **restrictions document** (DTD or Schema)



XML: validity

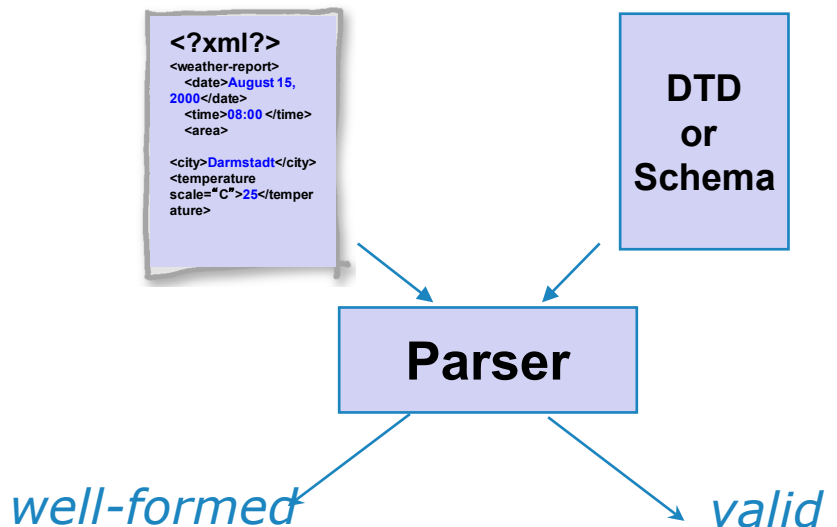
- Two ways of defining restrictions/rules for XML:

1. DTD (Document Type Definition):

- 1st generation: based on SGML,
- syntax not XML, few data types, ...

2. XML Schema:

- XML syntax,
- more data types,
- more restrictions,
- more complex



Schema languages

- Define the valid structure (grammar) of a set of XML documents (a XML application)
- Initially DTD (Document Type Definition):
 - Simple but limited
- Now XML Schema
 - Higher expressiveness, but very complex

Example with a DTD

```
<?xml version="1.0"?>
```

```
<!DOCTYPE film "film.dtd">
```

```
<films>
```

```
  <film>
```

```
    <title id="1">
```

```
      Tootsie
```

```
    </title>
```

```
    <genre>&COM;</genre>
```

```
    <year>1982</year>
```

```
  </film>
```

```
</films>
```

```
<!DOCTYPE film [
```

```
  <!ENTITY COM "Comedy">
```

```
  <!ENTITY SF "Science Fiction">
```

```
  <!ELEMENT films (film+)>
```

```
  <!ELEMENT film (title,genre,year)>
```

```
    <!ELEMENT title (#PCDATA)>
```

```
      <!ATTLIST title id ID>
```

```
    <!ELEMENT genre (#PCDATA)>
```

```
    <!ELEMENT year (#PCDATA)>
```

```
]>
```

Example with Schema

```
<?xml version="1.0"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  targetNamespace="http://www.films.org" xmlns="http://www.films.org">
  <xsd:element name="films">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="film" type="filmType" maxOccurs="unbounded"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
  <xsd:complexType name="filmType">
    <xsd:sequence>
      <xsd:element name="title" type="xsd:string"/>
      <xsd:element name="genre" type="xsd:string"/>
      <xsd:element name="year" type="xsd:string"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:schema>
```

Example with Schema

```
<?xml version="1.0"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  targetNamespace="http://www.films.org" xmlns="http://www.films.org">
  <xsd:element name="films"> Root element
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="film" type="filmType" maxOccurs="unbounded"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
  <xsd:complexType name="filmType"> Root element type definition,
    <xsd:sequence> includes sub-elements
      <xsd:element name="title" type="xsd:string"/>
      <xsd:element name="genre" type="xsd:string"/>
      <xsd:element name="year" type="xsd:string"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:schema>
```

Example of instance

```
<?xml version="1.0" encoding="UTF-8"?>
<films xmlns="http://www.films.org"          Schema reference
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:import namespace="http://www.films.org"
    schemaLocation="./films.xsd"/> Physical file location
  <film>
    <title>Tootsie</title>
    <genre>Comedy</genre>
    <year>1982</year>
  </film>
</films>
```

No entity definition nor id attribute

XML Schema & namespaces

`<films xmlns="http://www.films.org"` (1)

`xmlns:xsd="http://www.w3.org/2001/XMLSchema">` (2)

`<xsd:import namespace="http://www.films.org"`
`schemaLocation="./films.xsd"/>` (3)

- 1) Names without prefix define *default* namespace
"http://www.films.org"
- 2) Names with prefix "xsd" (import element) belong to the namespace "http://www.w3.org/2001/XMLSchema"
- 3) The schema corresponding to the namespace
"http://www.films.org" is in physical file "films.xsd" in the same directory as the xml file

XML Schema: Types

Simple Types (Datatypes) – Primitive

string	<i>any Unicode string</i>
boolean	true, false, 1, 0
decimal	3.1415
float	6.02214199E23
double	42E970
dateTime	2004-09-26T16:29:00-05:00
time	16:29:00-05:00
date	2004-09-26
hexBinary	48656c6c6f0a
base64Binary	SGVsbG8K
anyURI	http://www.brics.dk/ixwt/
QName	rcp:recipe, recipe
...	

XML Schema: Types

Derivation of Simple Types – Restriction

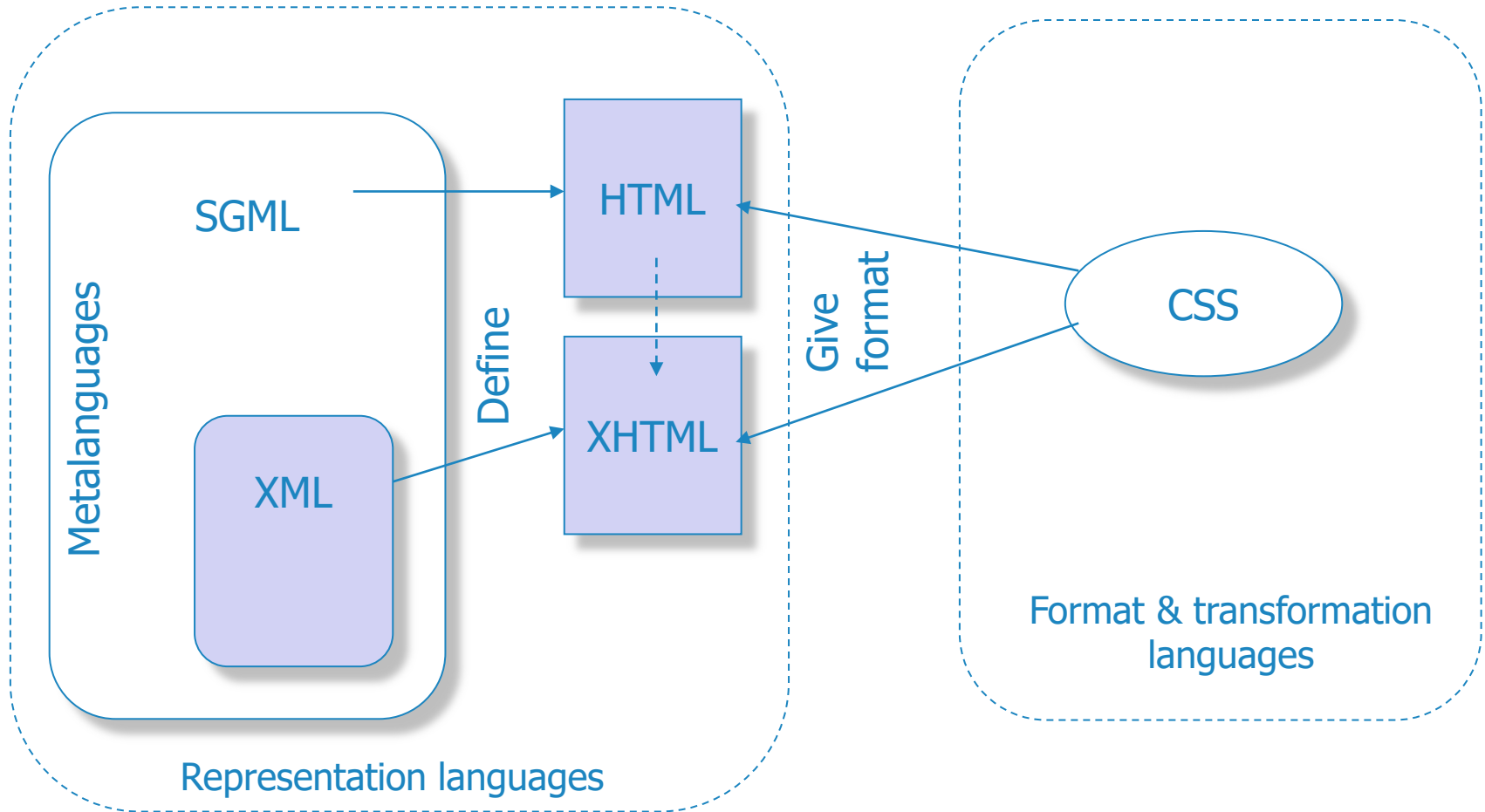
Constraining facets:

- length
- minLength
- maxLength
- pattern
- enumeration
- whitespace
- maxInclusive
- maxExclusive
- minInclusive
- minExclusive
- totalDigits
- fractionDigits

XML: Attribute definition

- How to define attributes in XML Schema?
 - `<xs:attribute name="xxx" type="yyy"/>`
- Where?
 - `<xs:element name="product">`
 `<xs:complexType>`
 `<xs:attribute name="prodid" type="xs:positiveInteger"/>`
 `</xs:complexType>`
 `</xs:element>`
- How is it represented?
 - `<product prodid="1345" />`

Languages relationships



Aplicacions Distribuïdes

Silvia Llorente
silvia.llorente@upc.edu

Distributed Multimedia Applications Group (DMAG)
Departament d'Arquitectura de Computadors (DAC)